TEAM SNACK OVERFLOW

TIVIDY

One workflow engine. Any industry.

COS 214 Final Project demonstration · University of Pretoria



Aidan du Bruyn · Connor Human · Ishmael Ngwasheng · Jay Macaskill
Keval Govind · Ruan van Tonder · Thabo Ngwabi

What TIVIDY is



WHAT IT DOES

Runs a business process step by step: who does which task, in what order, by when, and what happens when something goes wrong.



HOW IT WORKS

A workflow is described once as a template. Every run is its own copy that reacts to user actions, deadlines and outcomes.



THE PROBLEM IT SOLVES

Every industry needs the same process machinery. Rebuilding it for each one is slow and buggy, so we built it once and let industries plug in.

Two industries on one engine: Hudson (bank transaction disputes) · Warped (telecom service activation) · plus a generic document-approval demo pack. **No engine code knows any of them.**

How a workflow runs, in five steps

1

DEFINE

An industry's Director builds the template: stages, tasks, routing and deadlines.

2

START

createInstance() deep-clones the template into a private WorkflowInstance.

3

ACT

Each user action is a Command. The item's state decides if it is legal.

4

REACT

Every change is an event. FlowController routes, assigns and runs automatic steps.

5

FINISH

Complete, cancel, or expire/escalate when a deadline passes. All of it is in the audit log.

Simulated time: a ManualClock and advanceTime(seconds) let us show a deadline lapsing live, and test it in milliseconds.

What we learned before designing

WfMC glossary

A workflow management coalition is a shared vocabulary: a definition describes the process, an instance runs it. Work items go to participants.

Workflow Patterns (van der Aalst)

Sequence, parallel and conditional routes recur. Routing (what next) is separate from assignment (who).

BPMN 2.0.2

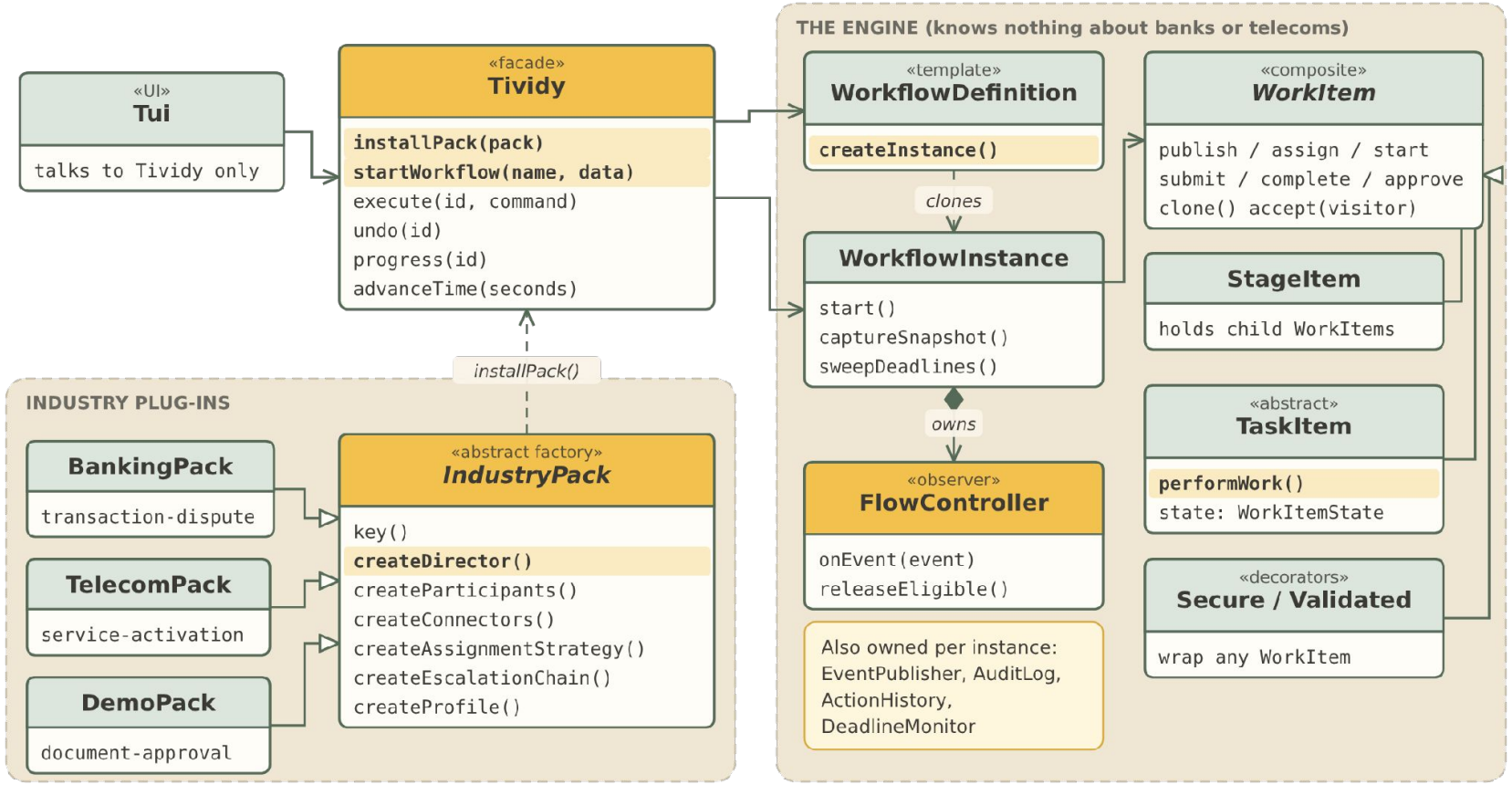
Progress changes when something happens: a task ends, a message arrives, a deadline passes. We model these as events.

Camunda docs

Human vs automatic tasks, instances keep their definition version, and history lets you inspect what happened.

Result: separate definitions, instances, work items, participants, decisions, events and history. Industries supply their own data and policies.

The whole system on one page



The UI knows one class

Tui talks only to Tividy, the facade.

Industries are packs

IndustryPack subclasses are the only place Hudson or Warped is named.

The engine is generic

Templates clone into instances; work items form a tree.

Industry packs configure the engine

HOW TO ADD ONE

1. Subclass IndustryPack.
2. Override createDirector(): the workflow.
3. Override only what differs: participants, connectors, assignment, escalation, profile.
4. Register it: installPack(pack).

0 engine files edited

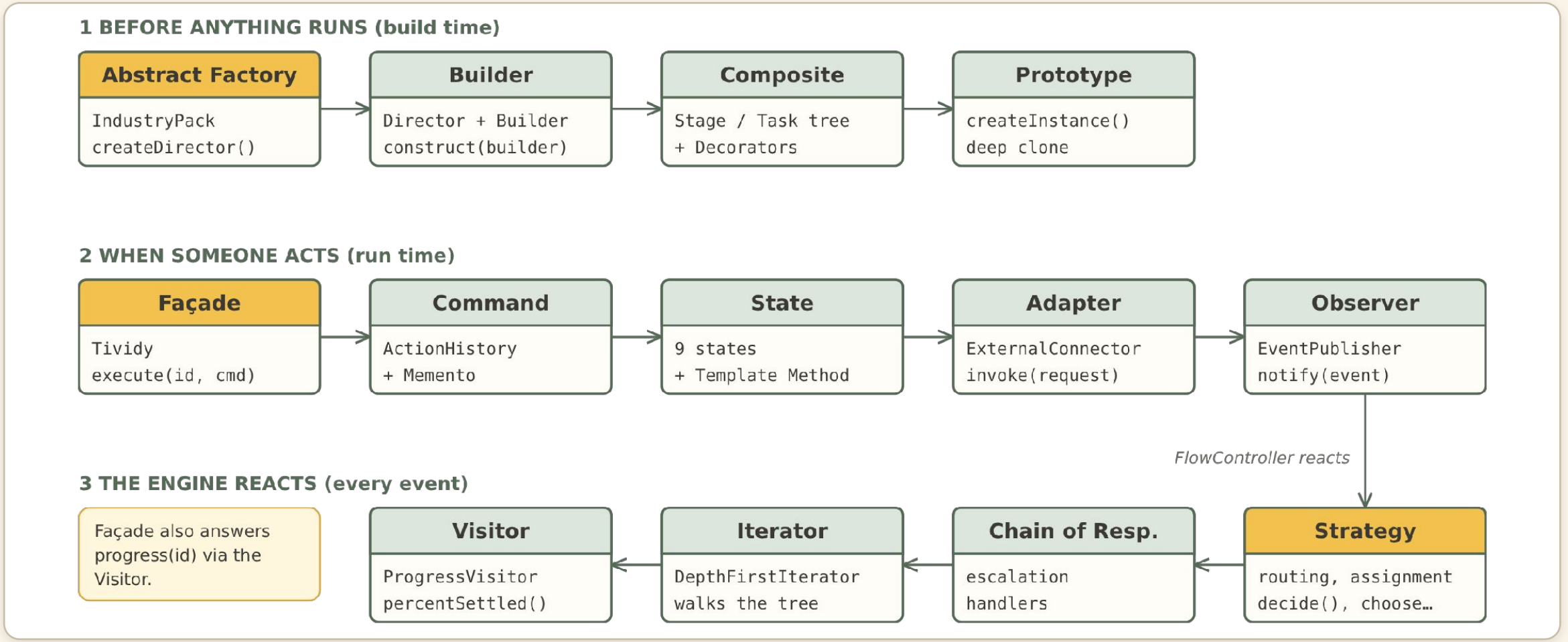
	Hudson	Warped	Demo
Workflow	DisputeDirector	ServiceActivationDirector	DocumentApprovalDirector
People	dispute-agent, analysts, merchant-rep	sales-agent, technicians, NOC	author, legal, finance, manager
Outside systems	core banking, card scheme	credit bureau, network provisioning	none (engine classes only)
Routing	outcome tables	outcome tables + install_type	outcome tables
Assignment	engine default (by role)	regional technician	by role
Escalation	senior-analyst → compliance-officer	network-engineer → noc-manager	role chain

02

The 16 design patterns

How they collaborate, then each one: why we chose it, what it does, how it helps.

How the 16 patterns work together



Read top to bottom: build the template → act on a private copy → the engine reacts to every event.

16 patterns, grouped by what they do

CREATIONAL · 3

Builder assembles workflows

Prototype clones templates per run

Abstract Factory industry packs

STRUCTURAL · 4

Composite stages and tasks as one tree

Decorator security, validation

Adapter outside vendors

Façade Tividy

BEHAVIOURAL · 9

State item lifecycle

Strategy routing, assignment

Observer events

Command user actions, undo

Memento snapshots

Template Method task execution

Chain of Resp. escalation

Iterator tree walk

Visitor progress reports

Builder: Assembling a workflow

WHY WE CHOSE IT

A workflow is a deep tree of stages, tasks, routing and deadlines. One giant constructor was never an option.

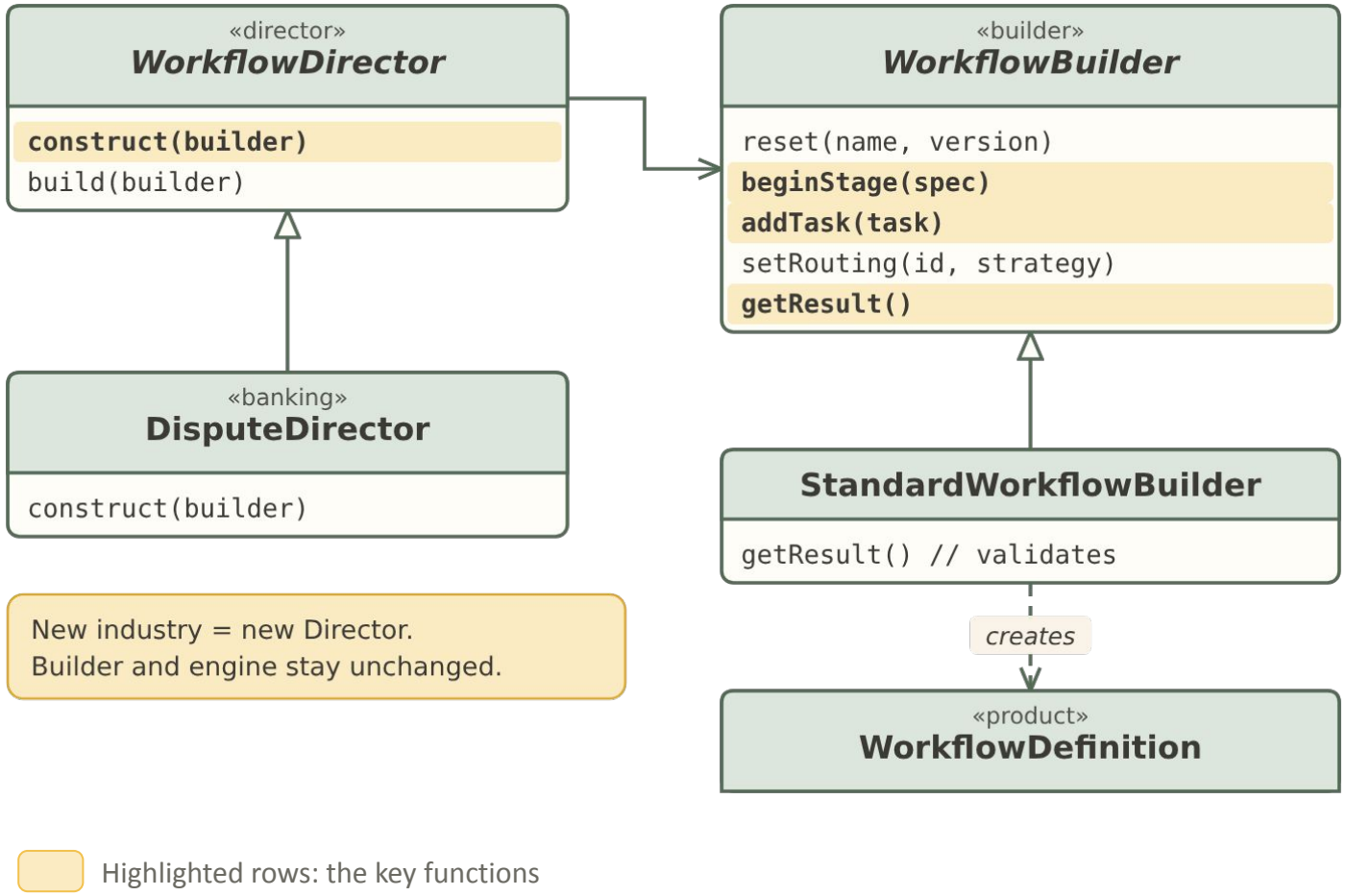
WHAT IT DOES

A Director calls beginStage, addTask and setRouting in order. The builder validates the result in getResult().

HOW IT HELPS

Each industry only describes its workflow. A new industry is a new Director; the engine does not change.

WORKS WITH Abstract Factory (supplies the Director), Composite (builds the tree), Prototype



Prototype: One private copy per run

WHY WE CHOSE IT

Every run needs its own copy of the workflow: same shape, separate state.

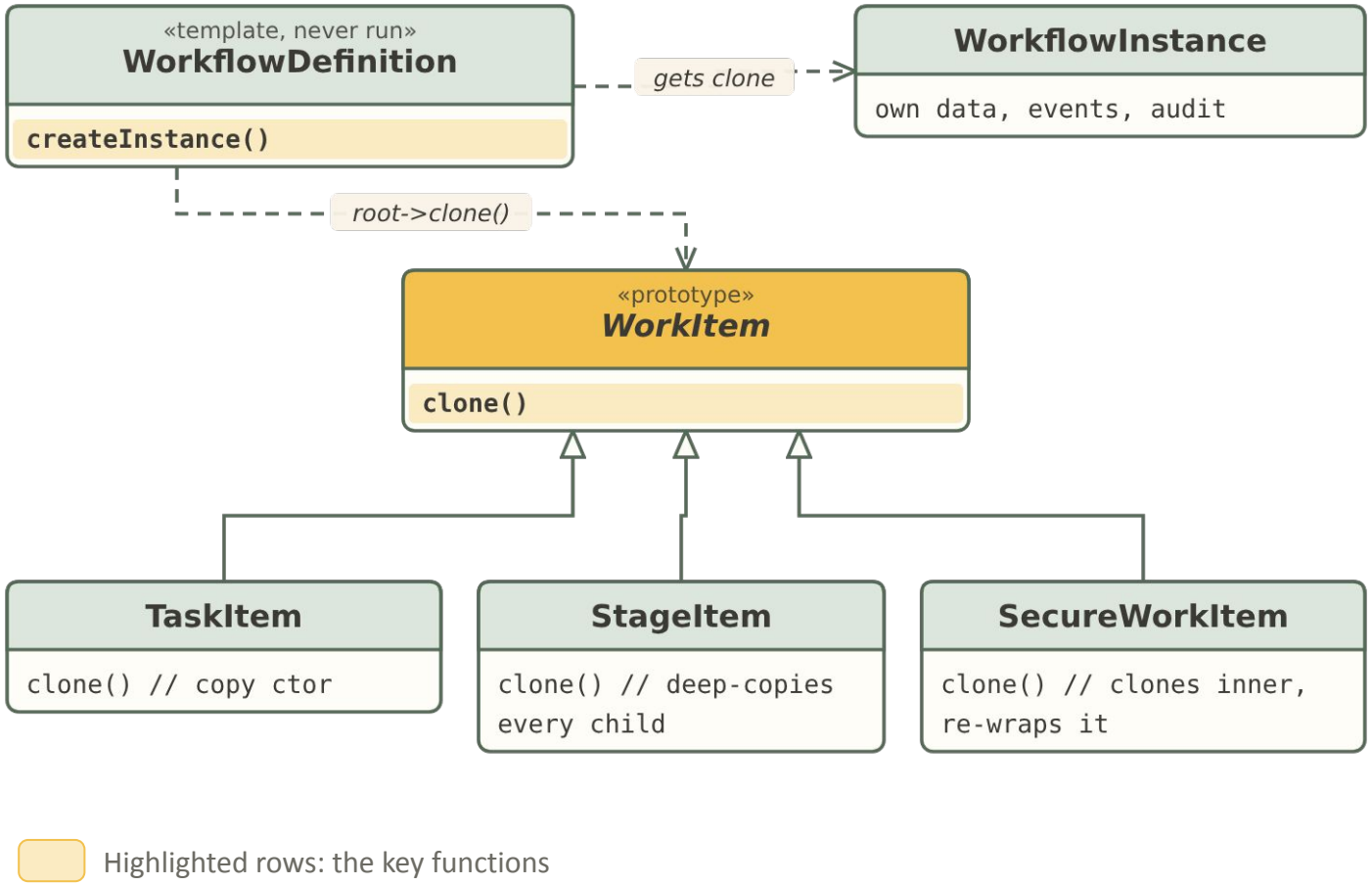
WHAT IT DOES

createInstance() calls clone() on the template tree. Stages deep-copy their children; decorators re-wrap their copy.

HOW IT HELPS

Runs never affect each other or the definition, and the engine never needs to know which task types exist.

WORKS WITH Builder (clones what it built), Composite, Decorator



Abstract Factory: Industry packs

WHY WE CHOSE IT

Each industry brings a whole family of parts that must match: workflow, people, connectors, policies.

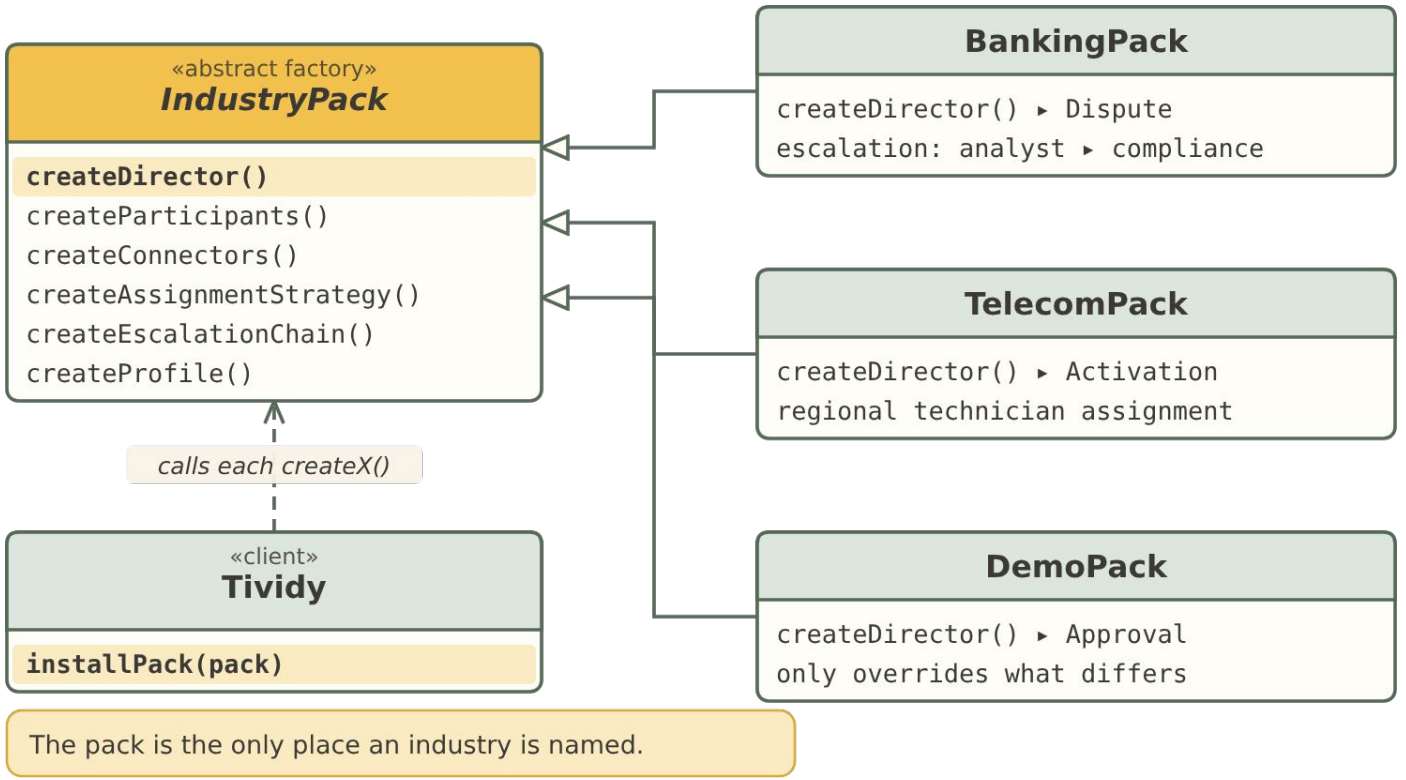
WHAT IT DOES

IndustryPack has one create method per part. installPack() calls them all; defaults mean a pack overrides only what differs.

HOW IT HELPS

Industry names appear only inside packs, so “same engine, two industries” is true by construction.

WORKS WITH Builder (createDirector), Strategy and Adapter (the pack supplies them)



Composite: Stages and tasks as one tree

WHY WE CHOSE IT

Work is nested: a stage holds tasks and other stages. The engine must treat both the same.

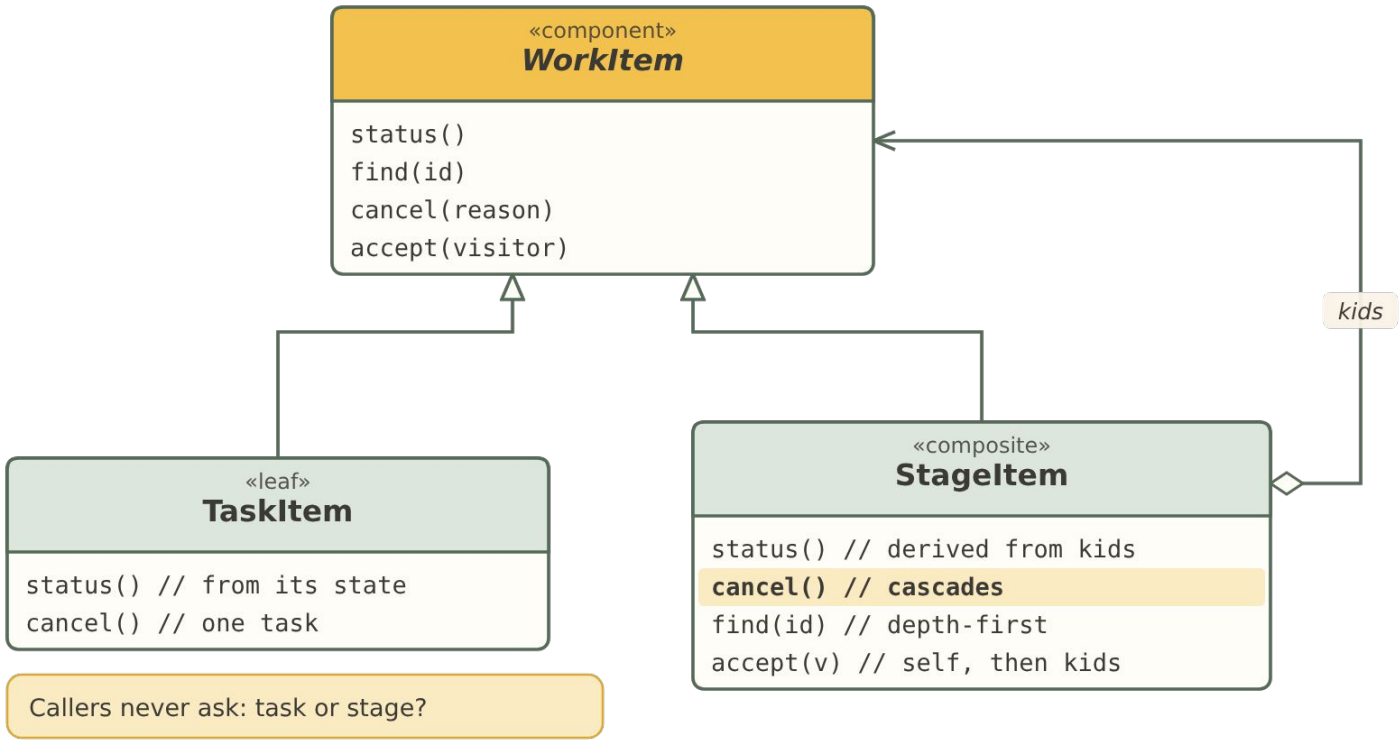
WHAT IT DOES

StageItem and TaskItem share the WorkItem interface. A stage's status comes from its children and cancel() cascades down.

HOW IT HELPS

No “is this a stage?” checks anywhere. Traversal, cancelling and reporting work on the whole tree.

WORKS WITH Decorator, Visitor, Iterator



Decorator: Security and validation, on demand

WHY WE CHOSE IT

Only some tasks need role checks or validation, and some need both. Subclasses would multiply endlessly.

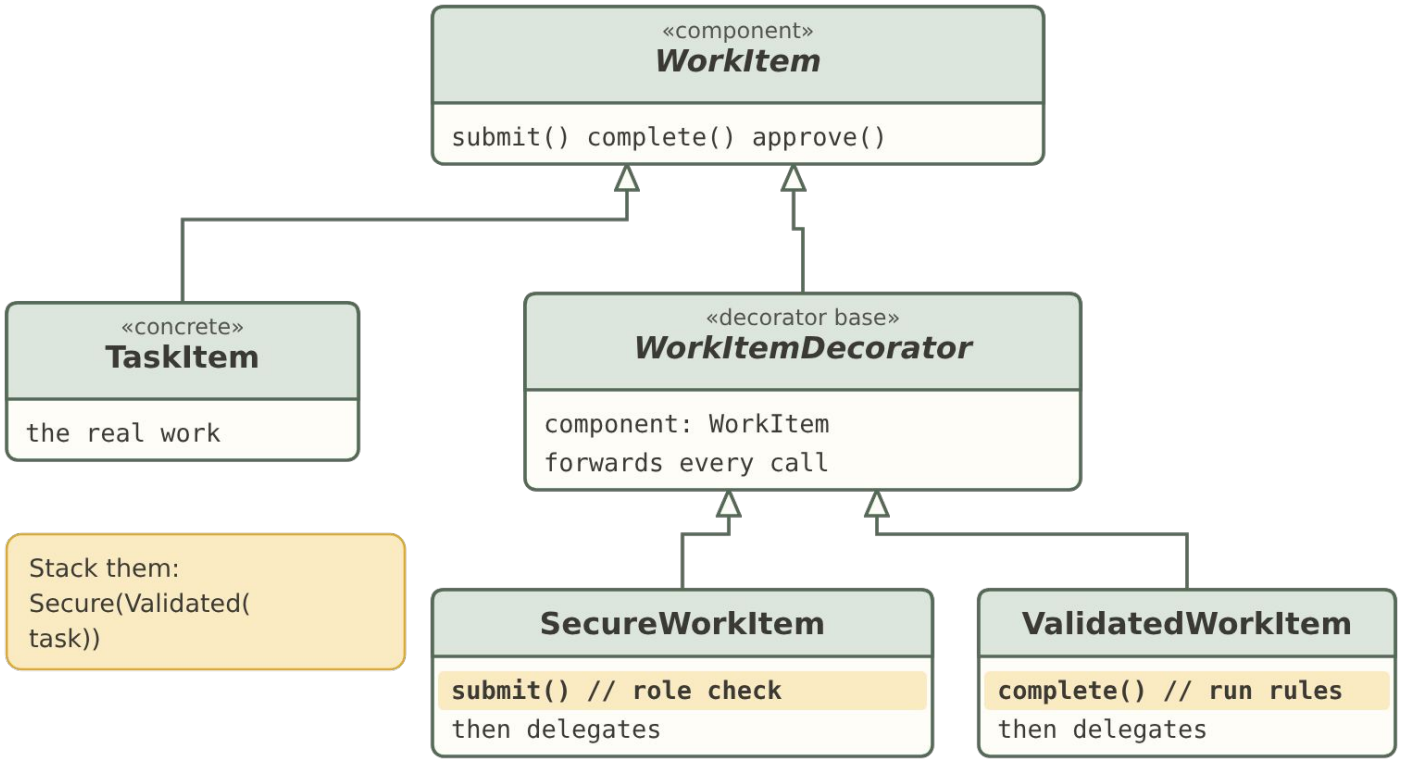
WHAT IT DOES

SecureWorkItem checks the actor's role, ValidatedWorkItem runs rules; each then forwards the call. They stack around any item.

HOW IT HELPS

Each concern lives in one class and is chosen per task by the industry's definition.

WORKS WITH Composite and Prototype (a clone re-wraps its inner item)



Adapter: Talking to outside systems

WHY WE CHOSE IT

Banks and telecoms use very different outside systems. The engine must not depend on any of them.

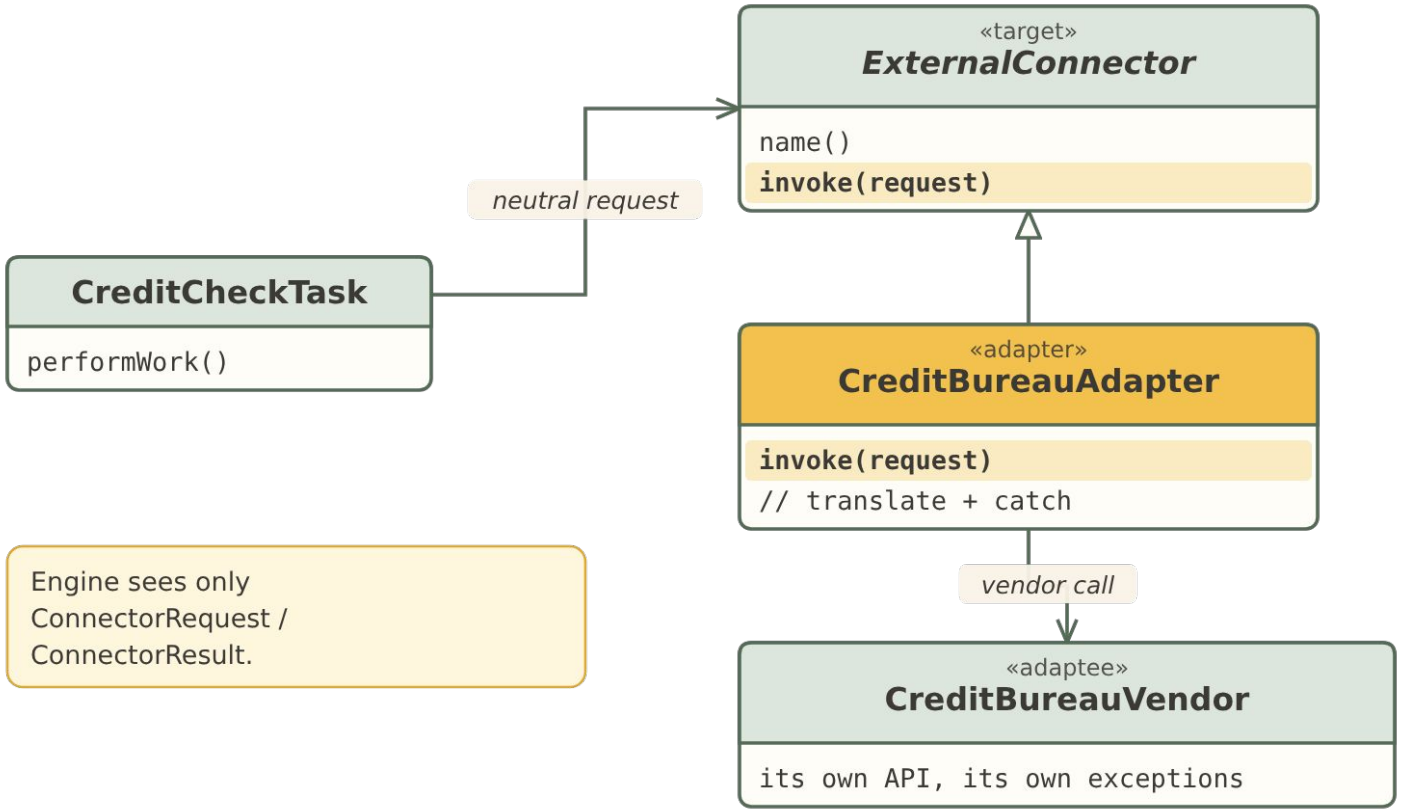
WHAT IT DOES

Tasks send a neutral ConnectorRequest. An adapter translates it to the vendor's API and turns exceptions into a ConnectorResult.

HOW IT HELPS

Swapping a vendor touches one adapter in one pack. No engine class changes.

WORKS WITH Template Method (tasks call connectors), Abstract Factory (packs supply them)



Façade: one front door for the UI

WHY WE CHOSE IT

A UI would otherwise juggle a dozen engine classes to install packs, start workflows and run actions.

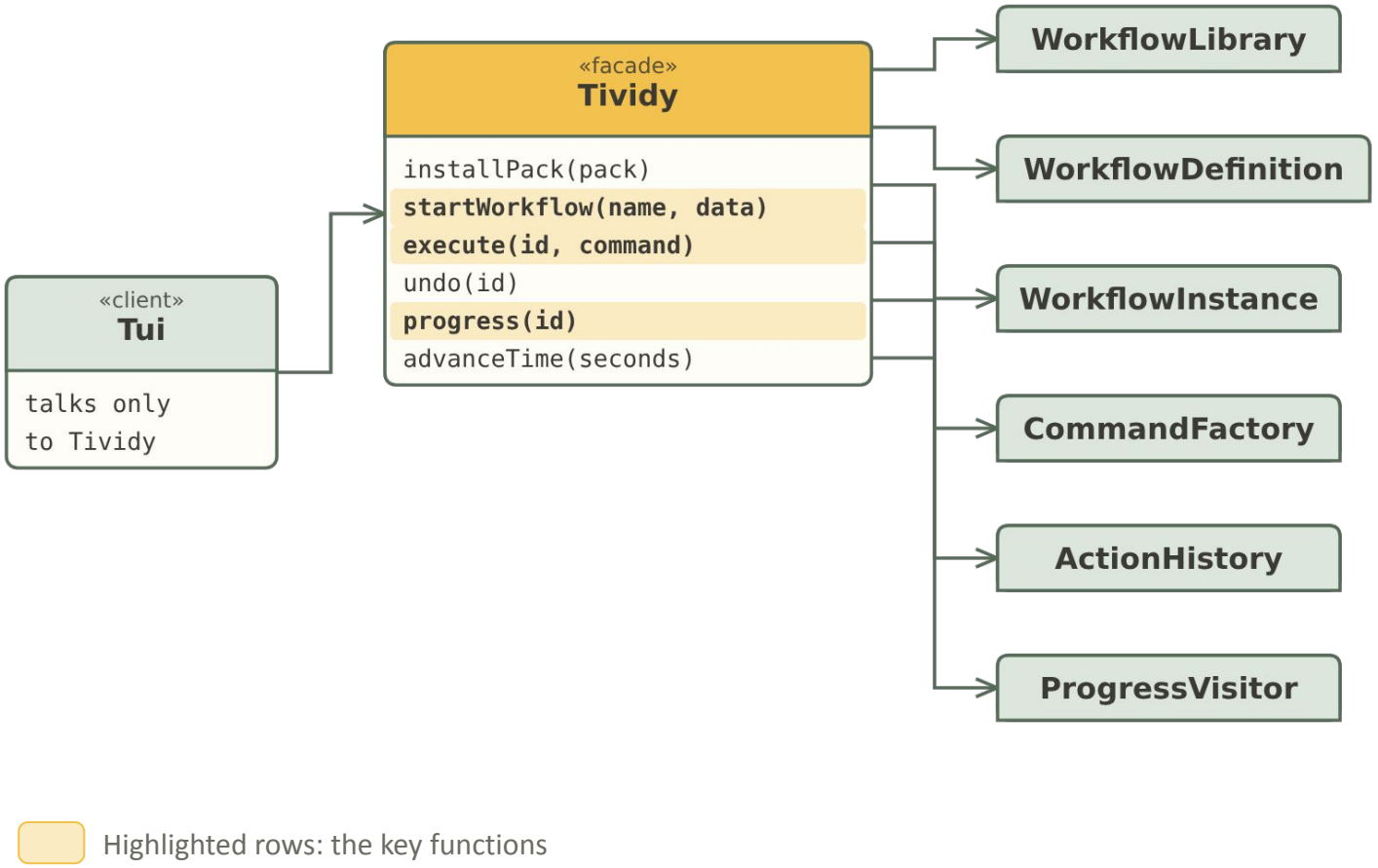
WHAT IT DOES

Tividy exposes a small set of calls: installPack, startWorkflow, execute, undo, progress, advanceTime.

HOW IT HELPS

The TUI only knows Tividy, so engine internals can change without touching the screens.

WORKS WITH Abstract Factory (installPack), Command, Visitor (progress)



State: The work item lifecycle

WHY WE CHOSE IT

A work item has nine statuses and most actions are illegal in most of them. A switch on status is what the spec forbids.

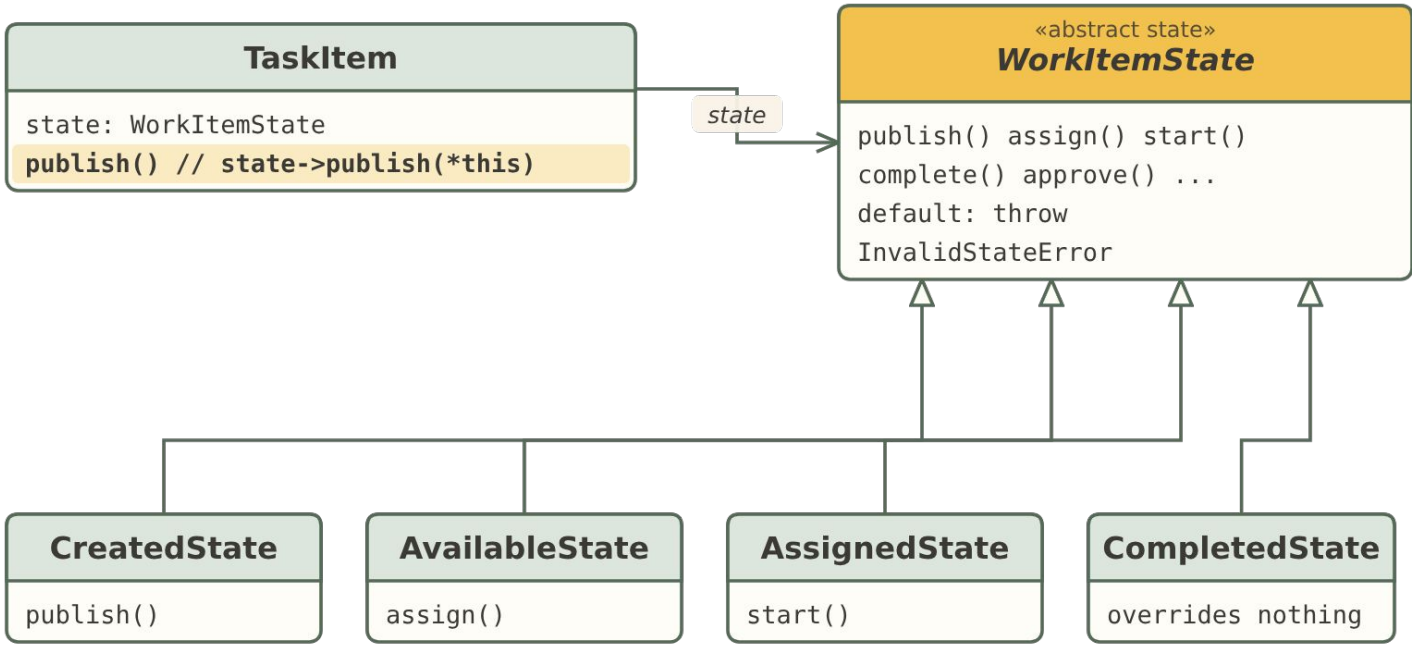
WHAT IT DOES

Each status is a state class that overrides only the actions legal for it. Everything else throws InvalidStateError.

HOW IT HELPS

Illegal actions are rejected automatically, and each transition is easy to test on its own.

WORKS WITH Template Method, Observer (events on each transition), Memento



9 shared, stateless singletons: Created, Available, Assigned, InProgress, UnderReview, Escalated, Completed, Cancelled, Skipped. No switch on status anywhere.

Highlighted rows: the key functions

Strategy: Routing and assignment

WHY WE CHOSE IT

What happens next, and who does the work, differ by industry. They also change independently.

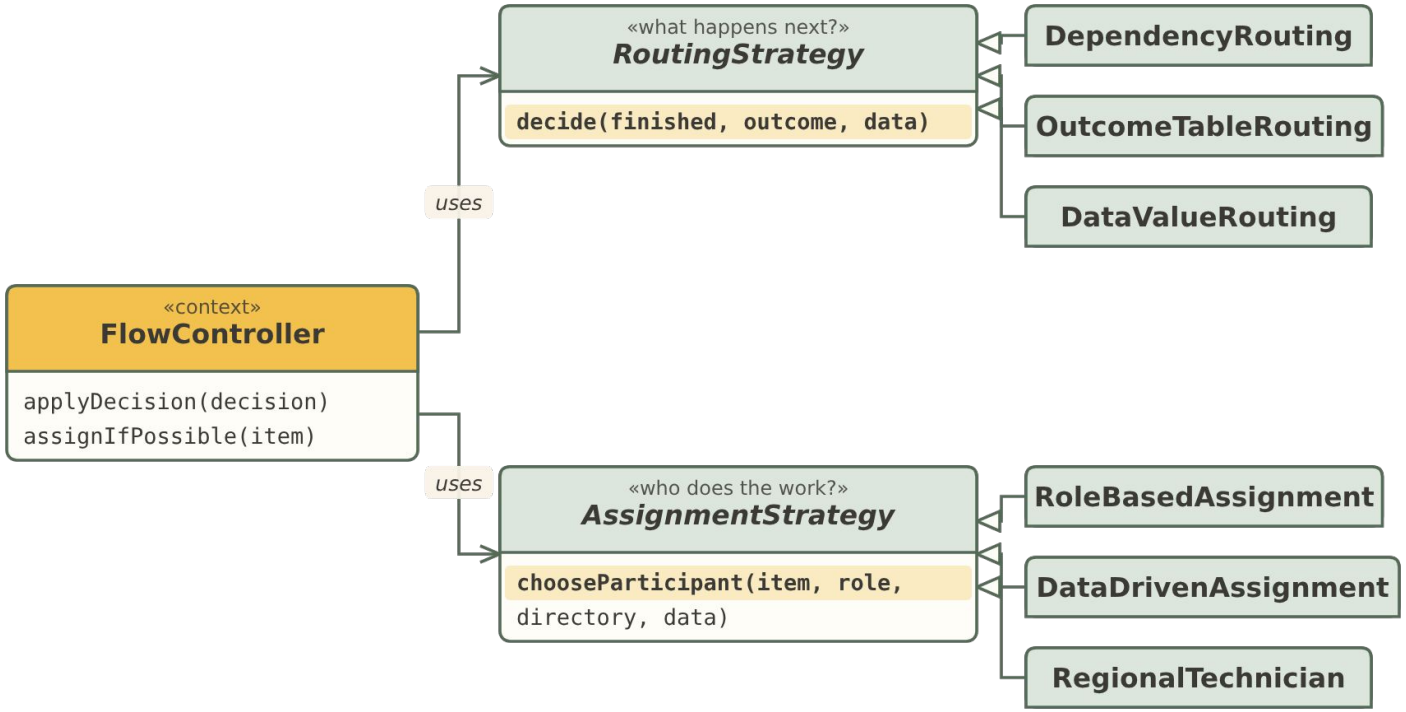
WHAT IT DOES

FlowController asks a RoutingStrategy what to activate or skip, and an AssignmentStrategy who should take the item.

HOW IT HELPS

Routing is configuration (tables), not code branches. New rules are new strategy objects.

WORKS WITH Observer (FlowController reacts), Abstract Factory (packs supply them)



Highlighted rows: the key functions

Observer: Reacting to every change

WHY WE CHOSE IT

Every state change must be audited, notified and routed. None of that belongs inside WorkItem.

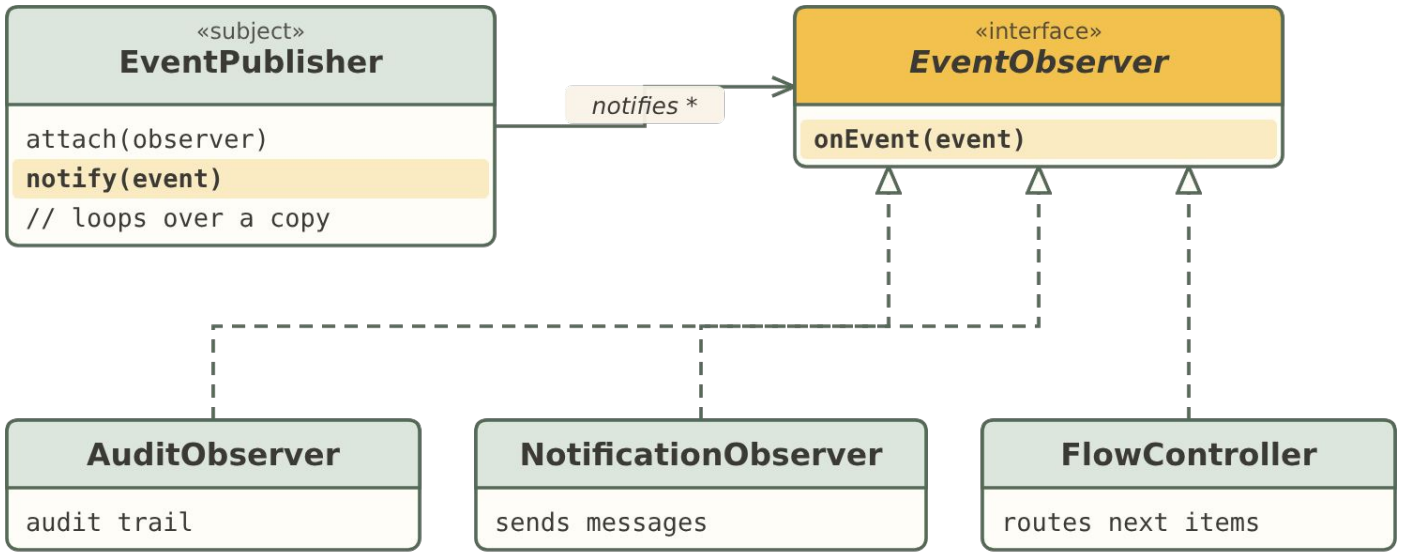
WHAT IT DOES

Each change publishes one WorkflowEvent. The publisher notifies every observer: audit, notifications, FlowController.

HOW IT HELPS

A new reaction is a new observer class. Each instance has its own publisher, so instances stay isolated.

WORKS WITH State, Strategy and Chain of Responsibility (all react to events)



One WorkflowEvent per state change. Each instance owns its own publisher.

Highlighted rows: the key functions

Command: User actions you can undo

WHY WE CHOSE IT

Every user action must be loggable, describable and undoable. A plain method call is none of those.

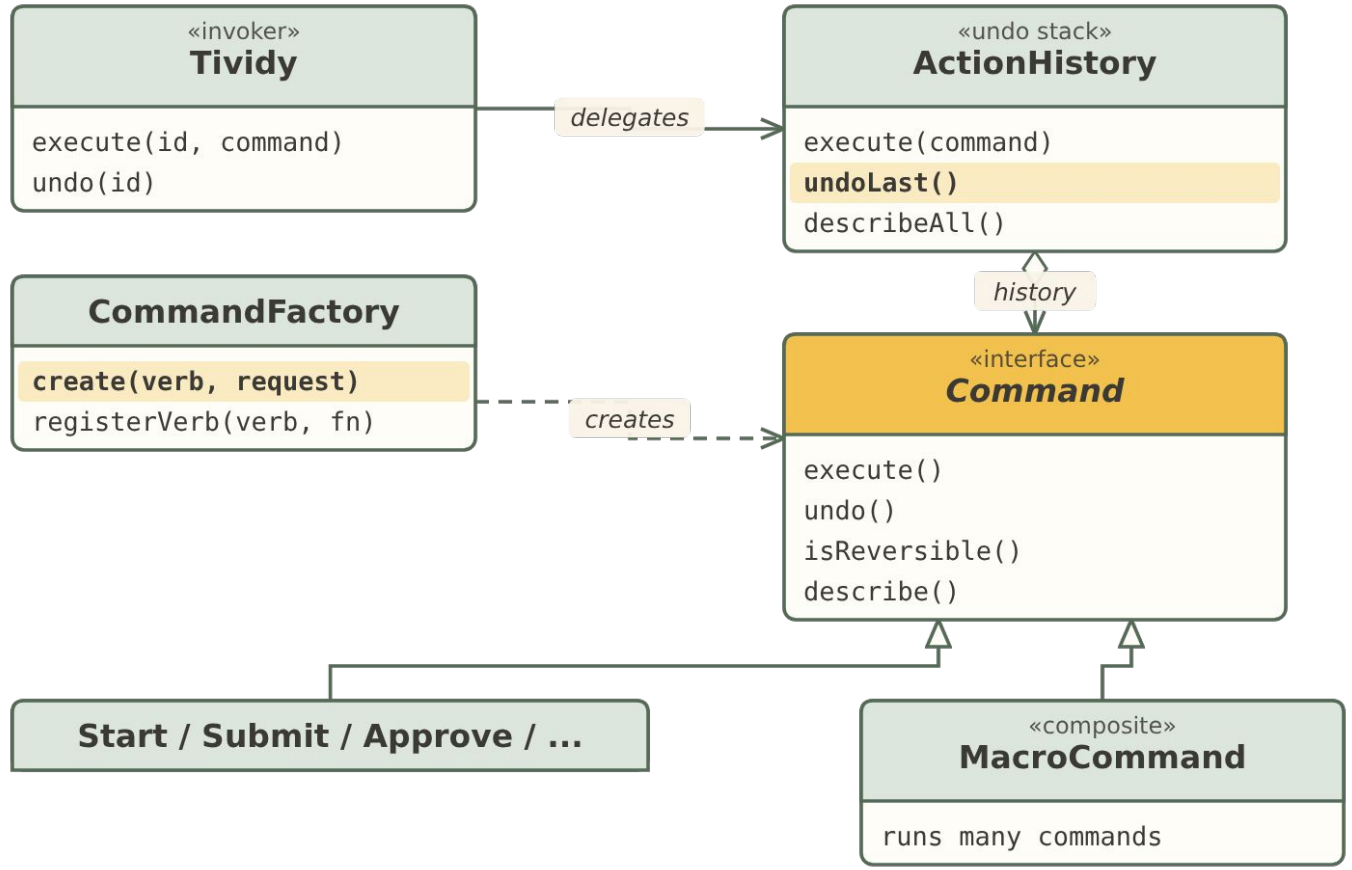
WHAT IT DOES

Tividy gets a command from CommandFactory and hands it to ActionHistory, which runs it and keeps it for undo.

HOW IT HELPS

Undo, audit text and macros come for free. New actions are added with registerVerb().

WORKS WITH Memento (snapshots), Façade, State, Composite (MacroCommand)



Memento: Snapshots for undo

WHY WE CHOSE IT

Undo must restore earlier state without exposing private fields to the history.

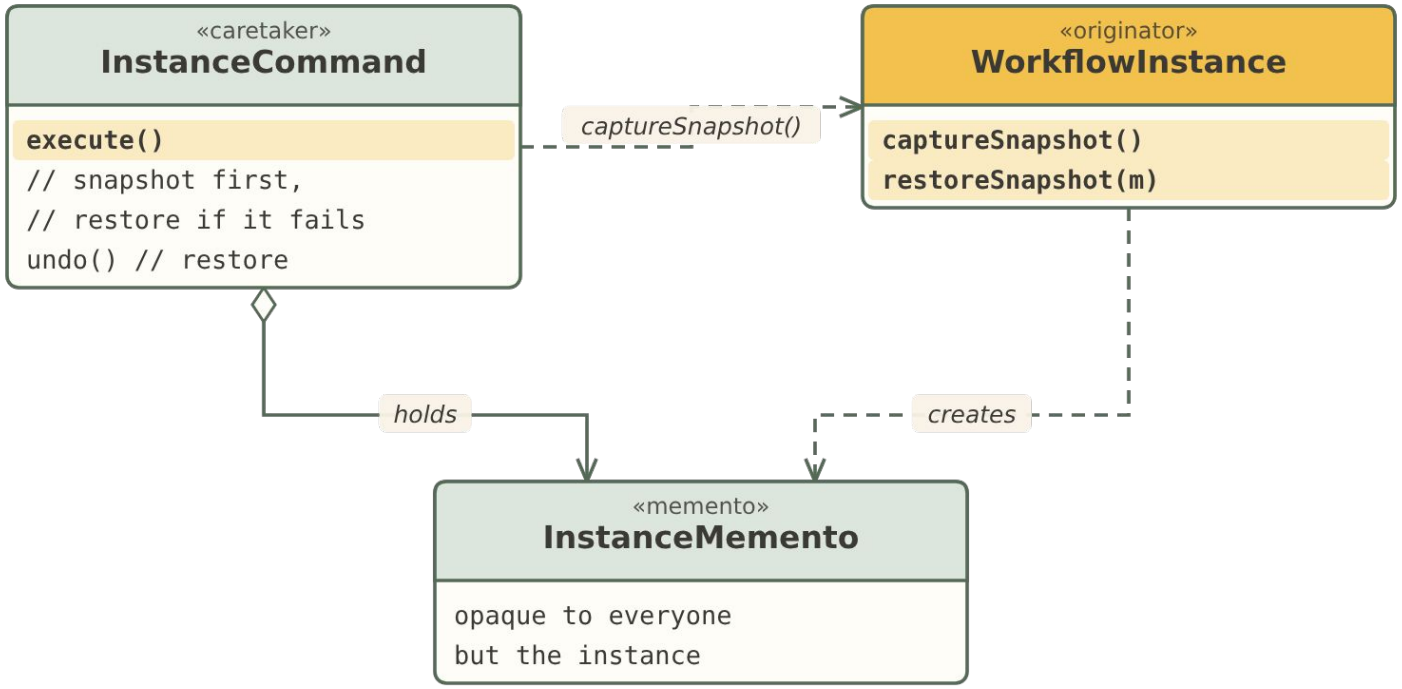
WHAT IT DOES

Before running, an InstanceCommand calls captureSnapshot(). If it fails or is undone, restoreSnapshot() puts things back.

HOW IT HELPS

The history holds opaque snapshots only, so encapsulation stays intact.

WORKS WITH Command (InstanceCommand takes the snapshot), State



History never sees the item's private fields.

Highlighted rows: the key functions

Template Method: Every task runs the same way

WHY WE CHOSE IT

Every task must run in the same order and always publish exactly one result. Only the actual work differs.

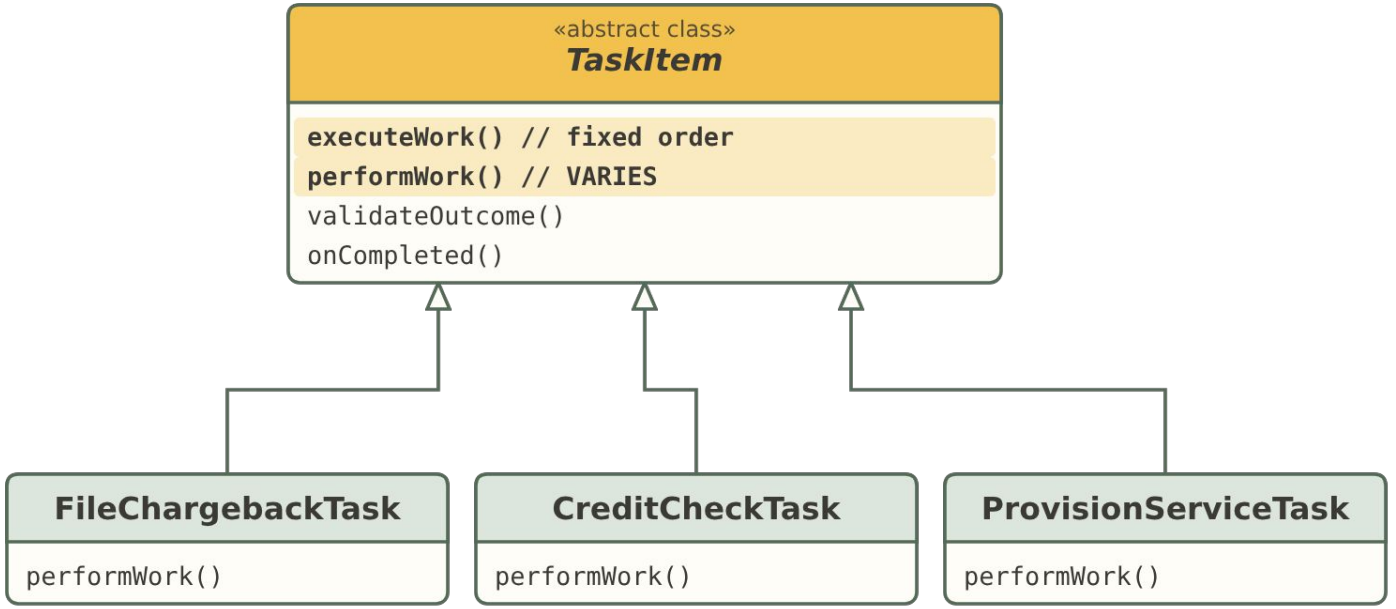
WHAT IT DOES

TaskItem::executeWork() fixes the steps. Subclasses fill in performWork(); a technical failure publishes ItemFailed instead.

HOW IT HELPS

No industry task can skip the audit trail, and a new task means writing one method.

WORKS WITH State (runs the transition), Adapter (outside calls), Observer



Fixed order: validate ▸ work ▸ record ▸ transition ▸ publish.
A failure publishes ItemFailed and never changes state.

Highlighted rows: the key functions

Chain of Responsibility: Escalation

WHY WE CHOSE IT

An escalated item must reach the right role for its level, without knowing who that is.

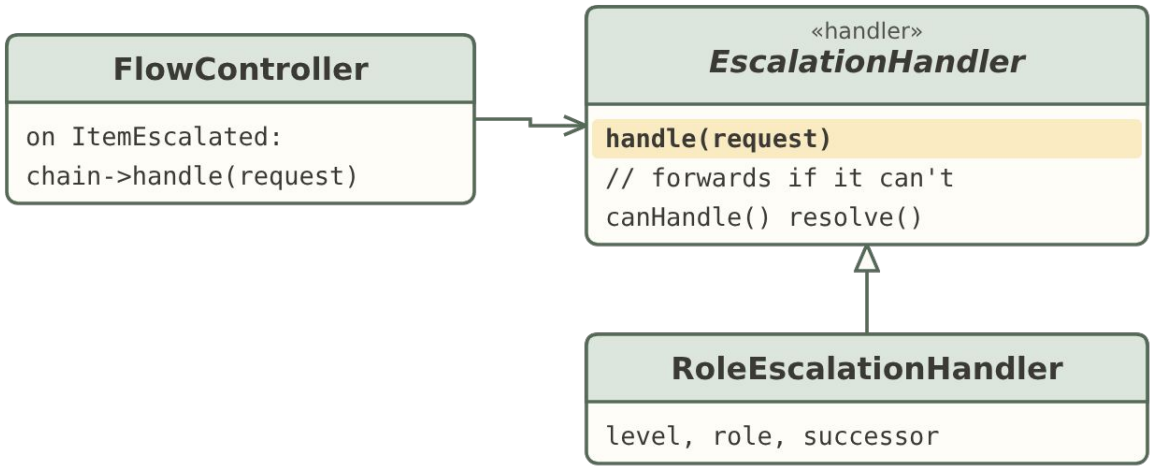
WHAT IT DOES

FlowController hands the request to the first handler. Each handler resolves its own level or passes it to its successor.

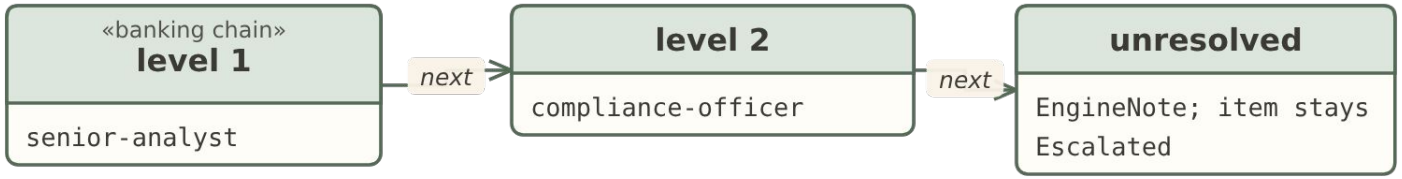
HOW IT HELPS

Banking and telecom reuse one handler class with different settings. If nobody resolves it, the item stays Escalated.

WORKS WITH Observer (ItemEscalated), Abstract Factory (the pack supplies the chain)



Two instances of the same class, configured:



Highlighted rows: the key functions

Iterator: Walking the tree

WHY WE CHOSE IT

Several algorithms walk the whole tree and must not depend on how children are stored.

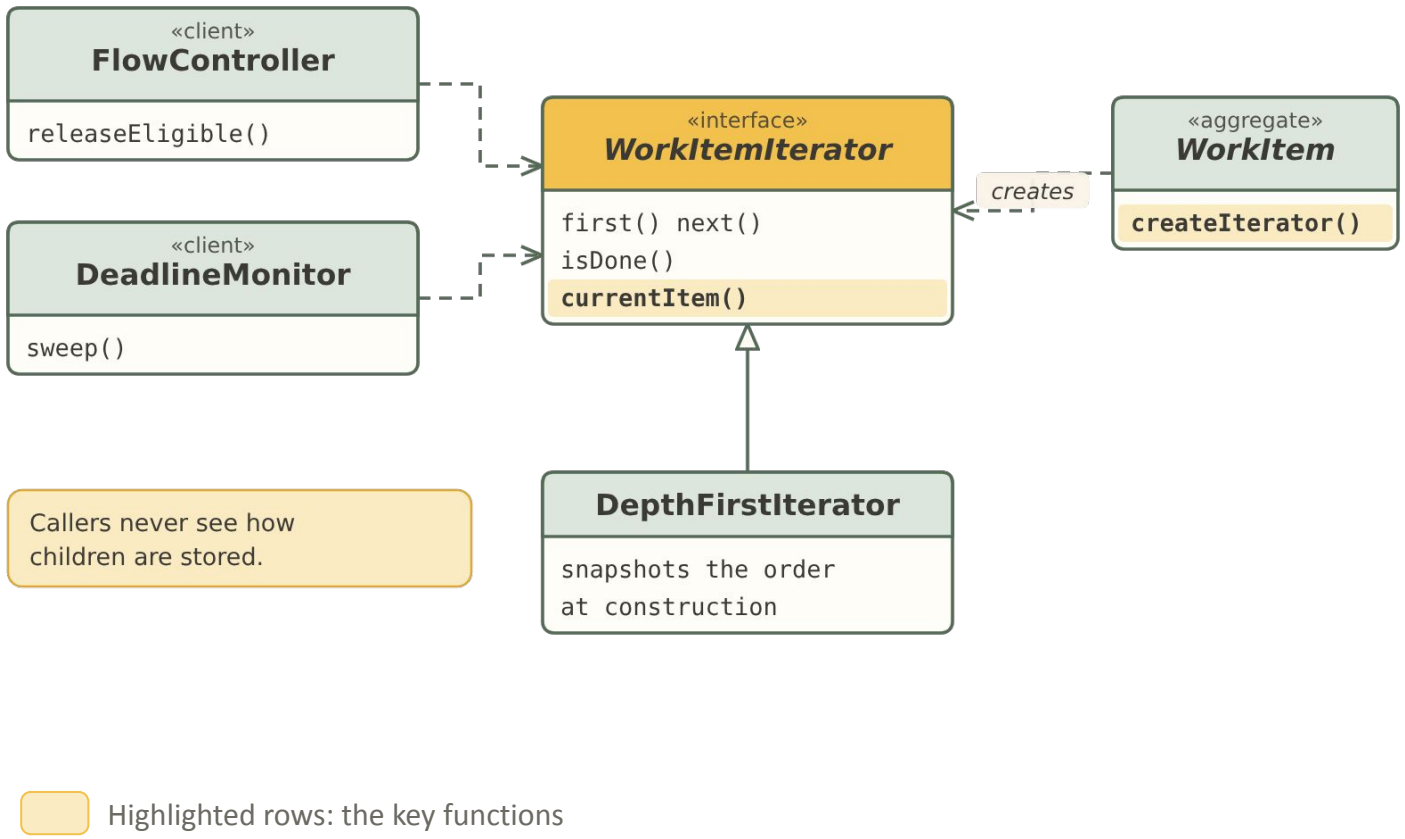
WHAT IT DOES

createIterator() returns a DepthFirstIterator that snapshots the traversal order: first, next, isDone, currentItem.

HOW IT HELPS

FlowController and DeadlineMonitor share one traversal, and storage stays private.

WORKS WITH Composite (the tree), Visitor



Visitor: Progress reports

WHY WE CHOSE IT

New reports over the tree keep appearing, but the item classes are stable.

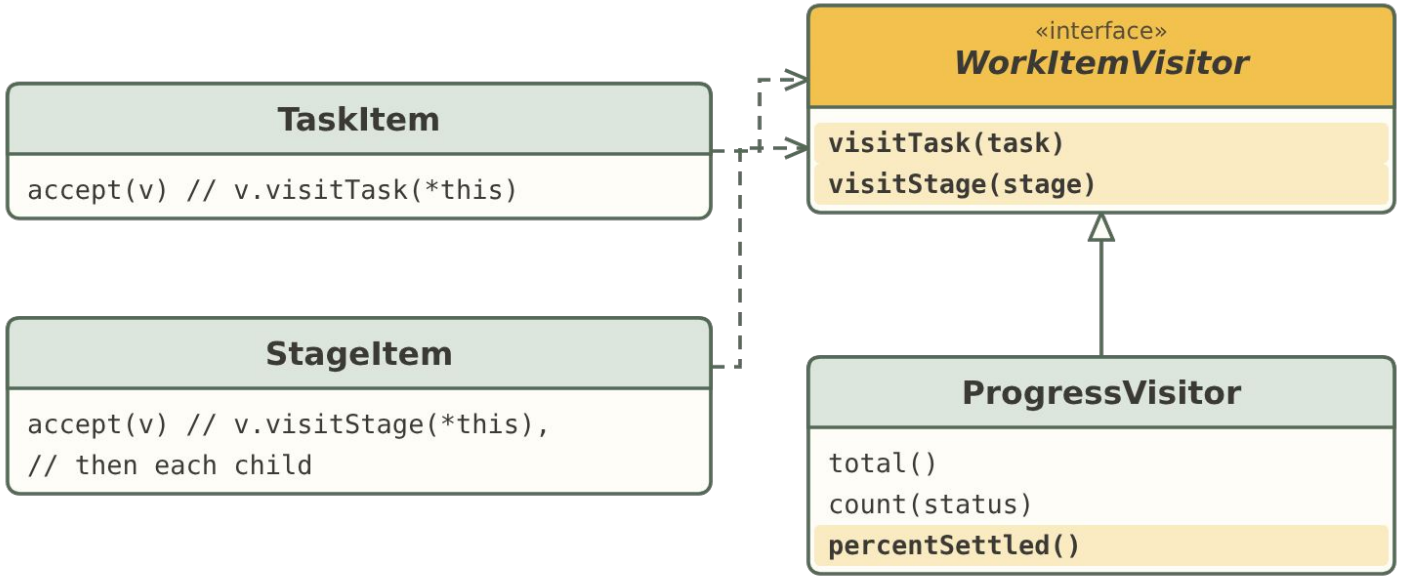
WHAT IT DOES

accept(visitor) calls visitTask or visitStage, then recurses. ProgressVisitor counts items by status.

HOW IT HELPS

A new report is a new visitor class. No item class is edited.

WORKS WITH Composite (the tree), Iterator, Façade (progress)



Tividy::progress(id) builds a ProgressVisitor, calls root().accept(v), then reads the counts. New report = new visitor, no item edits.

Highlighted rows: the key functions

Design rationale: six decisions that came from the research

1 Definition vs instance

One definition yields many independent instances, so a change to one run never affects another.

2 Routing vs assignment

What happens next and who does it vary independently, so each is its own configurable strategy.

3 Explicit lifecycle

Valid actions depend on a work item's state, so the lifecycle is modelled as states, not flags.

4 Events apart from work

Audit, notifications and routing all react to the same events, so none is wired into the task.

5 Behaviour without subclassing

Security and validation wrap an item instead of creating new task types.

6 History kept separately

An audit log records every action and outcome and never removes earlier entries.

Alternatives we rejected, and why

Prototype, not Factory Method

A factory per task type would force the engine to know every concrete task. clone() lets the template stay opaque, and a new task needs no engine edit.

One WorkItem interface, not role interfaces

Every engine client treats tasks and stages alike, so role interfaces would force downcasts at each call site. The logic lives in collaborators: states, context and structure.

State objects, not switch(status)

Nine statuses and eleven operations would pile about ninety cases into one switch. Each state class handles only its own, so it is testable and extensible.

In each case the rejected option would have put knowledge of every concrete class into the engine.

03

Industries and scenarios

Hudson and Warped on one engine, and the diagrams behind them.

Two industry workflows, one engine design

HUDSON · BANKING

`transaction-dispute`

- A customer disputes a card charge.
- Bank posts a provisional credit and files a chargeback.
- Merchant answers; an analyst decides; a senior analyst approves (four-eyes).
- Either side may appeal; then the ledger is settled.

Exception: merchant stays silent, the deadline lapses and the dispute is decided against them.

WARPED · TELECOM

`service-activation`

- A customer orders a mobile or fibre service.
- ID is verified and a credit check runs automatically.
- Self-install ships a device; technician install is scheduled by region.
- Service is provisioned and the customer confirms.

Exception: a failed or late step escalates network-engineer → noc-manager.

Same engine for both: states · routing · deadlines · undo · audit history

What the running system demonstrates

Start a workflow

Pick an industry and enter the case data. A private instance is cloned from the template.

Decisions and approvals

People act in their roles: assign, start, submit, approve or reject. Illegal actions are refused.

A deadline lapsing

The simulated clock advances so a missed deadline is decided or escalated on screen.

Undo

The last action is reversed from a snapshot, and the audit trail records it.

History and progress

Every instance has an audit trail and a live progress summary.

Two industries

Warped runs on the same engine as Hudson. Only its pack is swapped in.

Hudson: the transaction-dispute workflow

1. `lodge_dispute` — customer
2. `assess_eligibility` — dispute-agent
3. `issue_provisional_credit` — dispute-agent
4. `file_chargeback` — system
5. `merchant_response` — merchant-rep
6. `analyst_decision` — analyst + senior approves
7. `appeals` — either side may appeal
8. `settle` — system

Green steps run automatically (role: system). The rest are done by people.

ROUTING

assess_eligibility can end the dispute early if it is ineligible. The merchant conceding or going silent settles for the customer.

DECISION

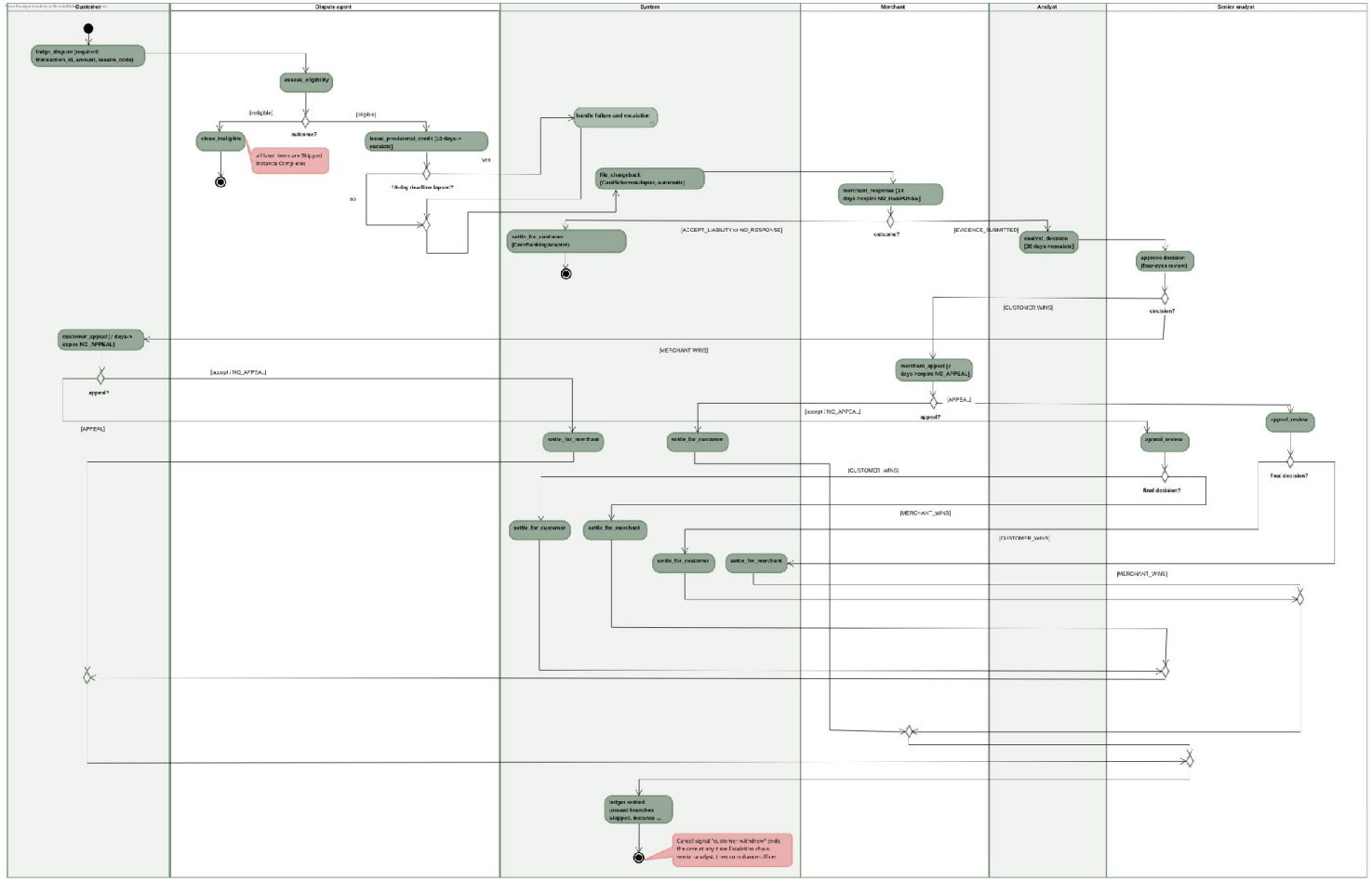
The analyst proposes, the senior analyst approves (four-eyes). The loser may appeal to a senior analyst for a final decision.

ONE SETTLEMENT

Exactly one settlement runs. Unused items are Skipped so the instance completes.

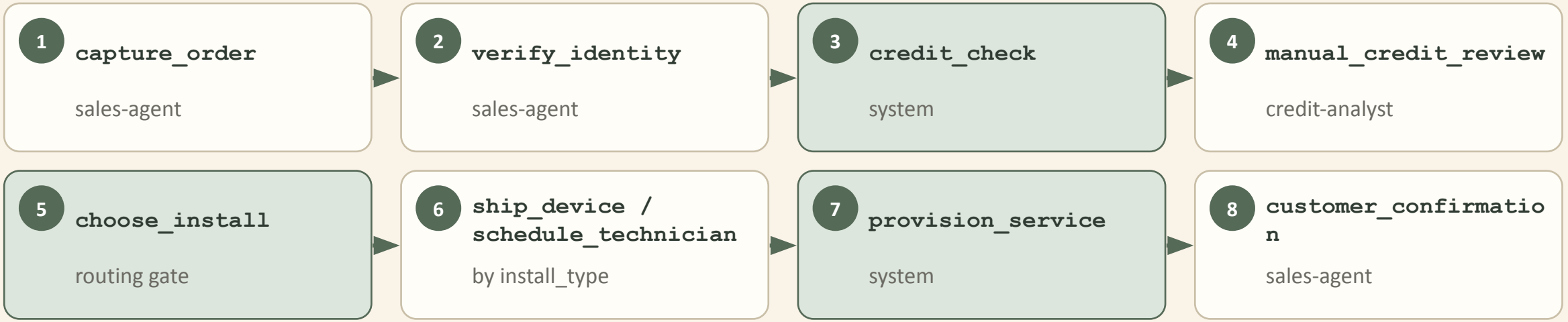
Exception path: if the merchant does not answer, the deadline lapses and the dispute is decided against the merchant by default.

Hudson: from lodging a dispute to one settlement



From lodging the dispute to exactly one settlement · Shows the early exit, the default decision and the appeal route

Warped: the service-activation workflow



Green steps run automatically (role: system). The rest are done by people.

CREDIT ROUTING

OK continues; REFER goes to a credit analyst; DECLINED notifies the customer and the order completes.

INSTALL ROUTING

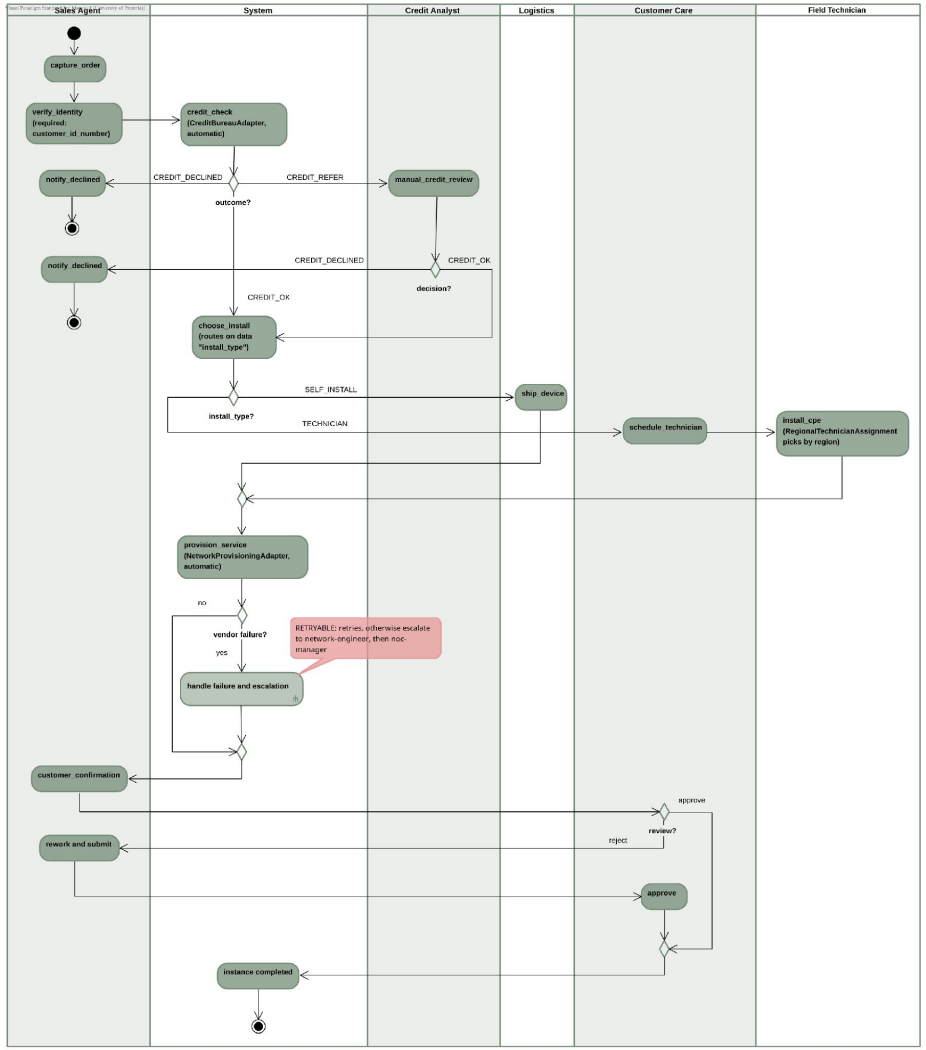
SELF_INSTALL ships a device. TECHNICIAN schedules a field technician, assigned by region.

SAME ENGINE

Same states, routing, deadlines, undo and audit as Hudson. Only the pack changed.

Exception path: if provisioning fails or runs late, the item escalates network-engineer → noc-manager through the chain of responsibility.

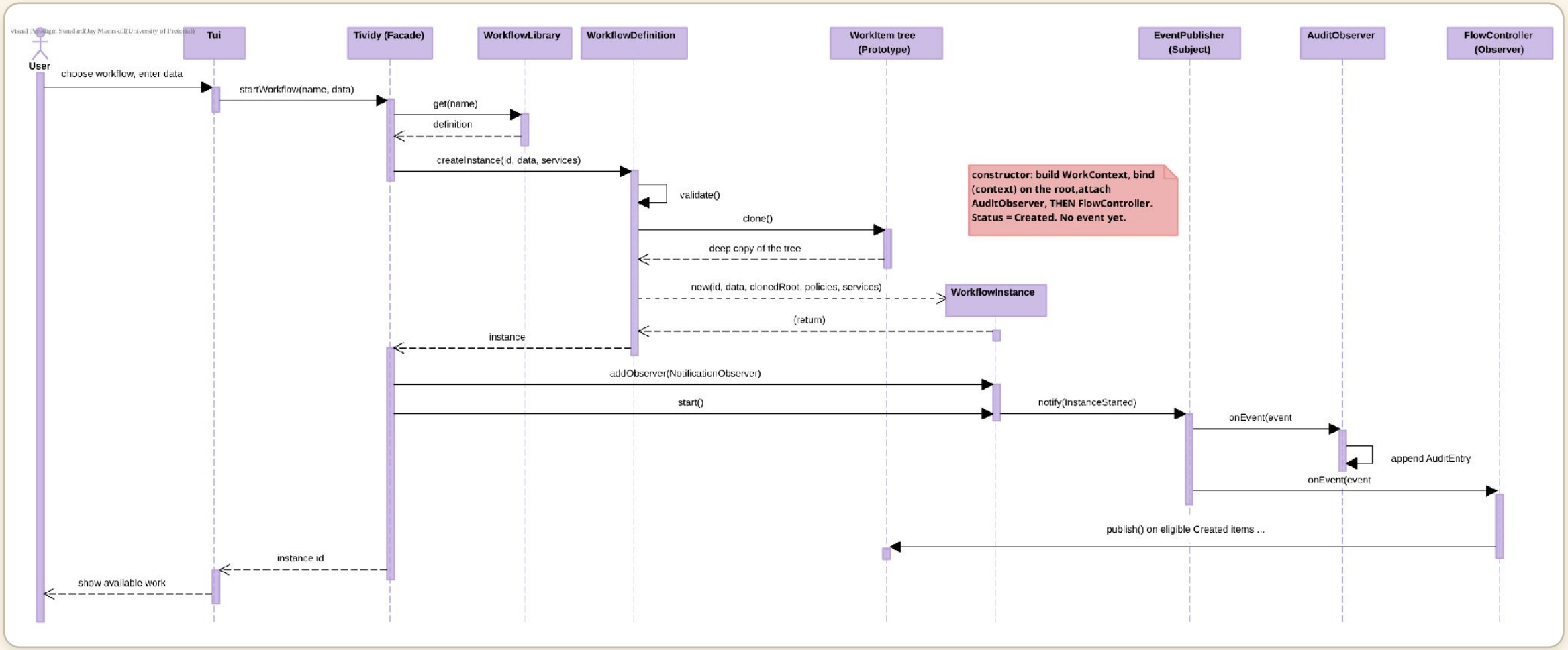
Warped: from order to a confirmed service



KEY POINTS

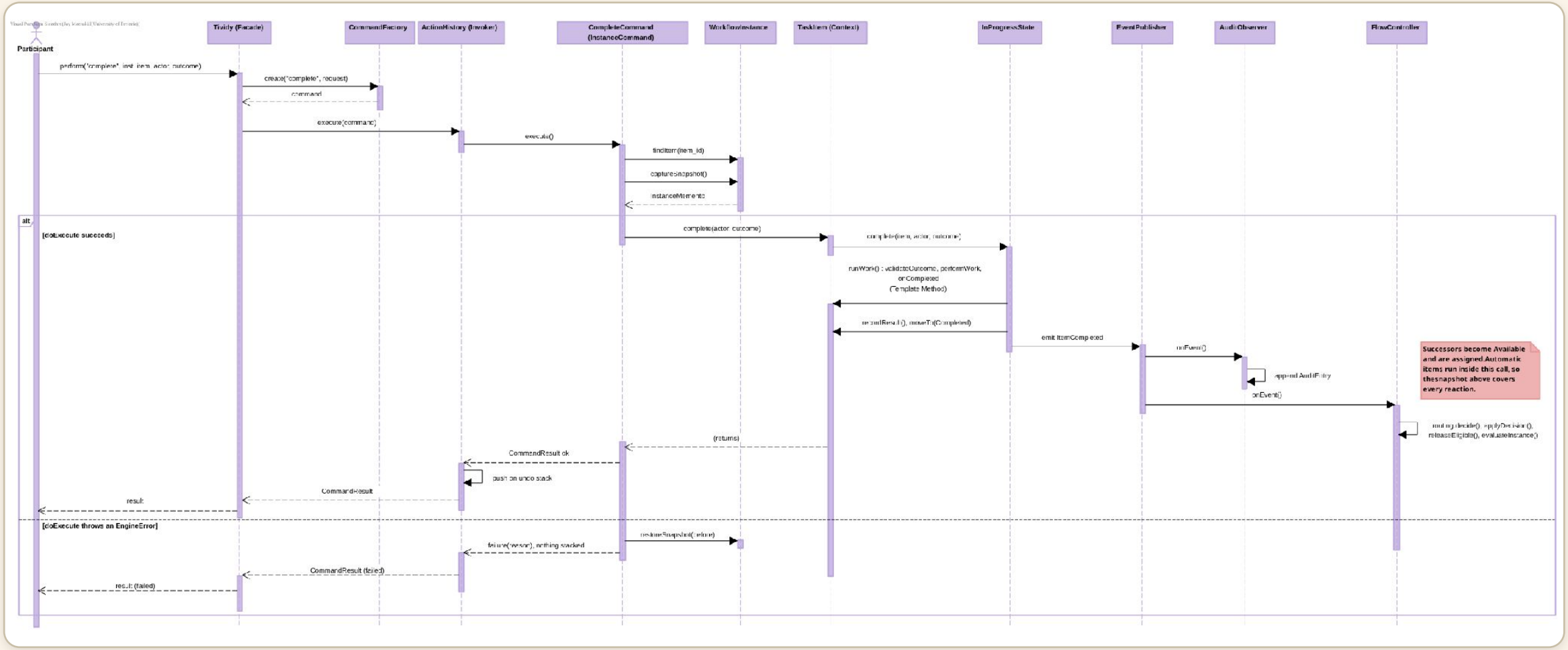
- Credit routing: OK, refer or declined
- Install routing: self-install or technician
- Provisioning, then customer confirmation

Starting a workflow



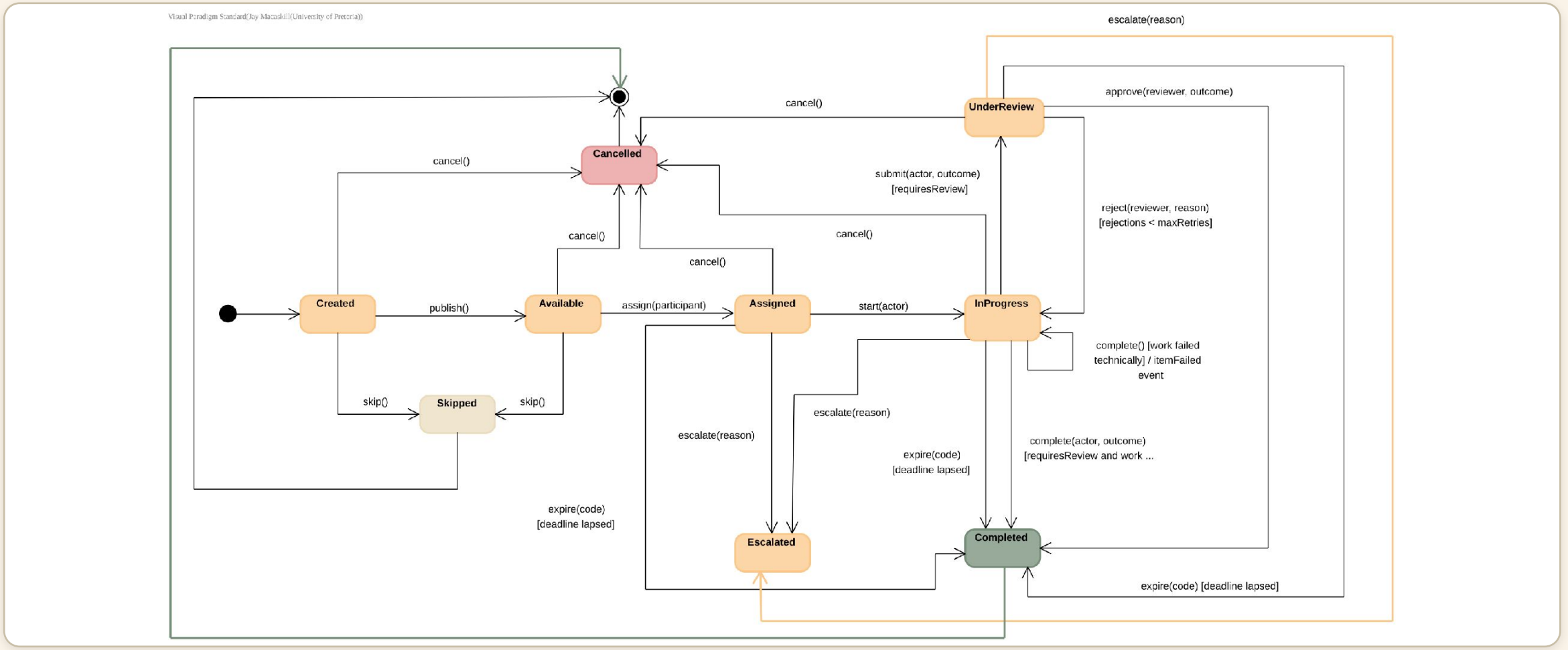
Tividy looks up the definition · createInstance() deep-clones the template · The instance starts and FlowController releases the first items

Taking a user action



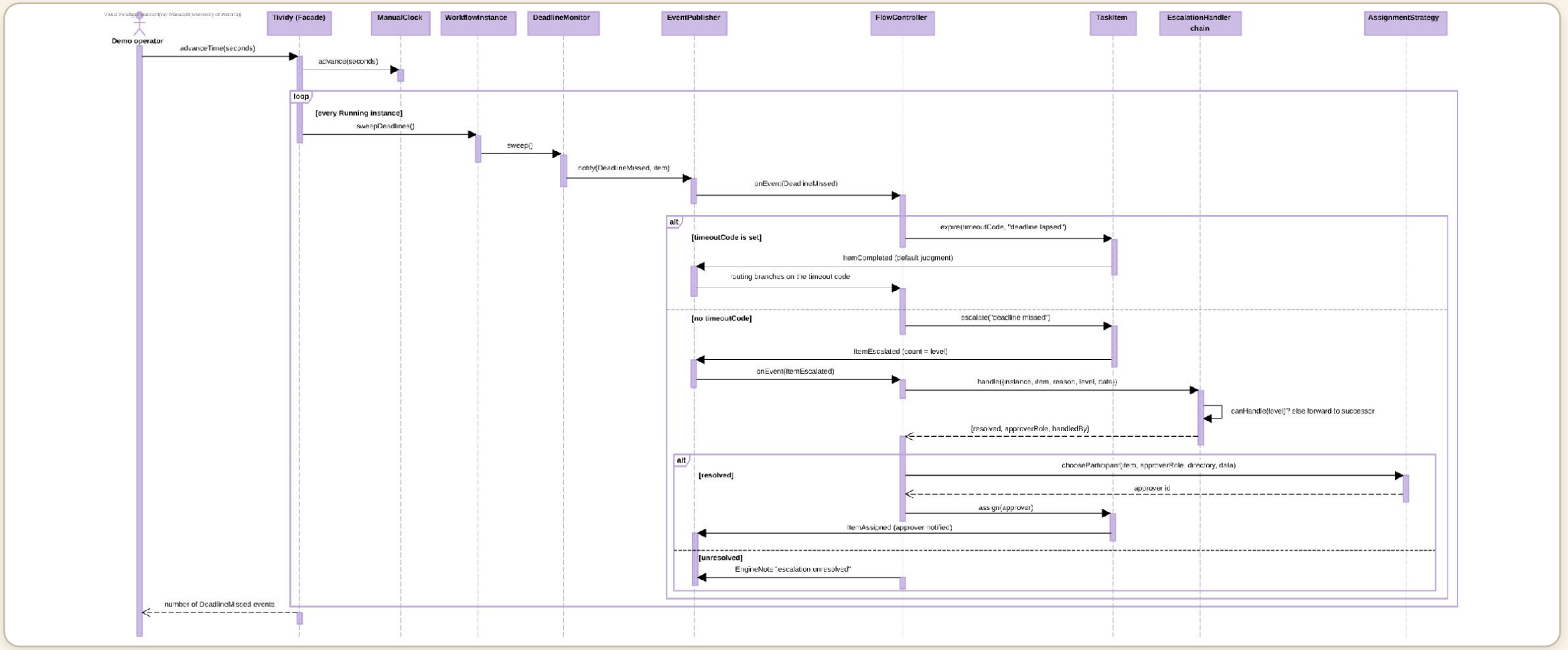
The action becomes a Command run through ActionHistory · The item's state validates it · Events fan out to audit, notifications and FlowController

The work item lifecycle



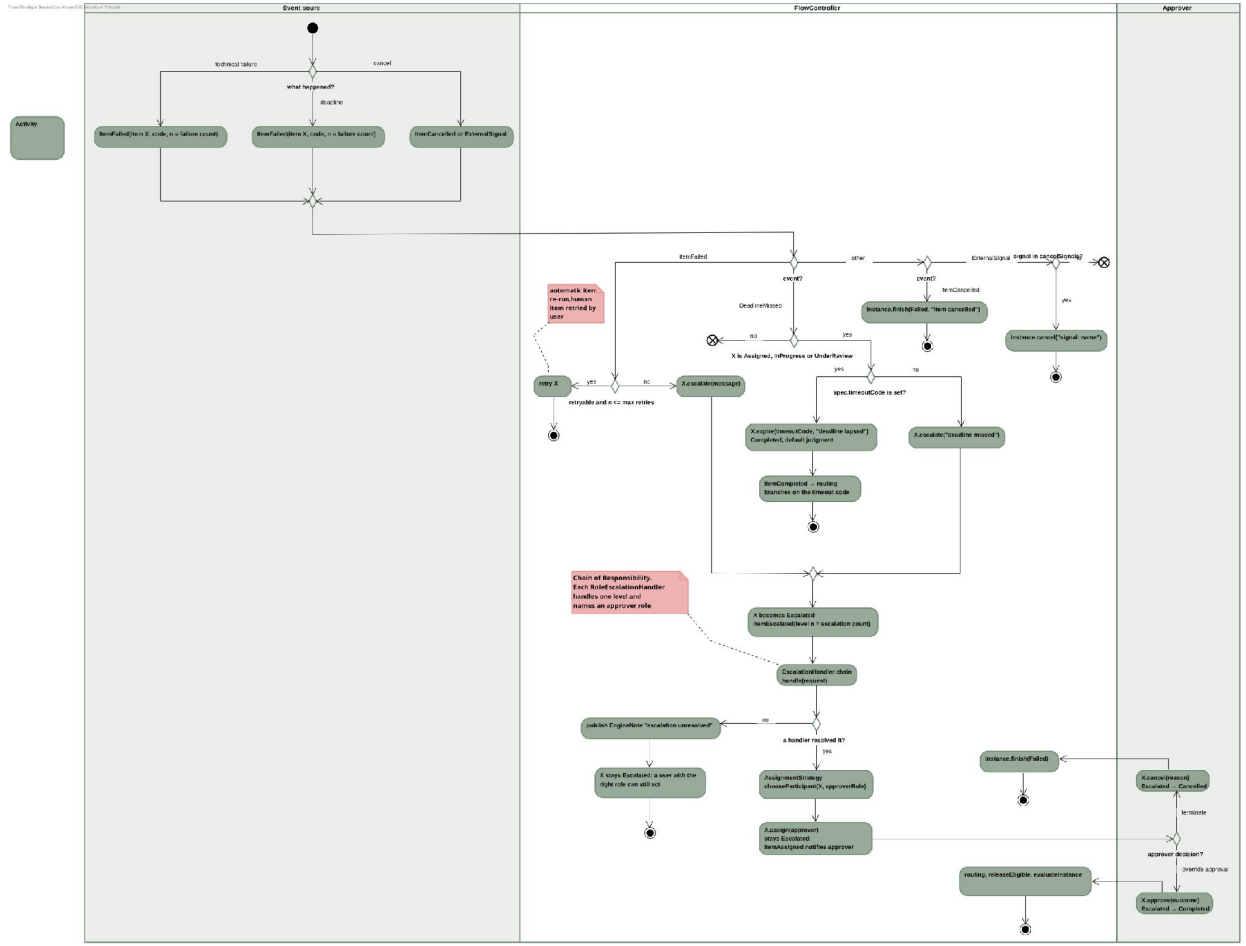
Nine statuses, eleven operations · Cancel is allowed from any non-terminal state · Illegal actions throw InvalidStateError

When a deadline lapses



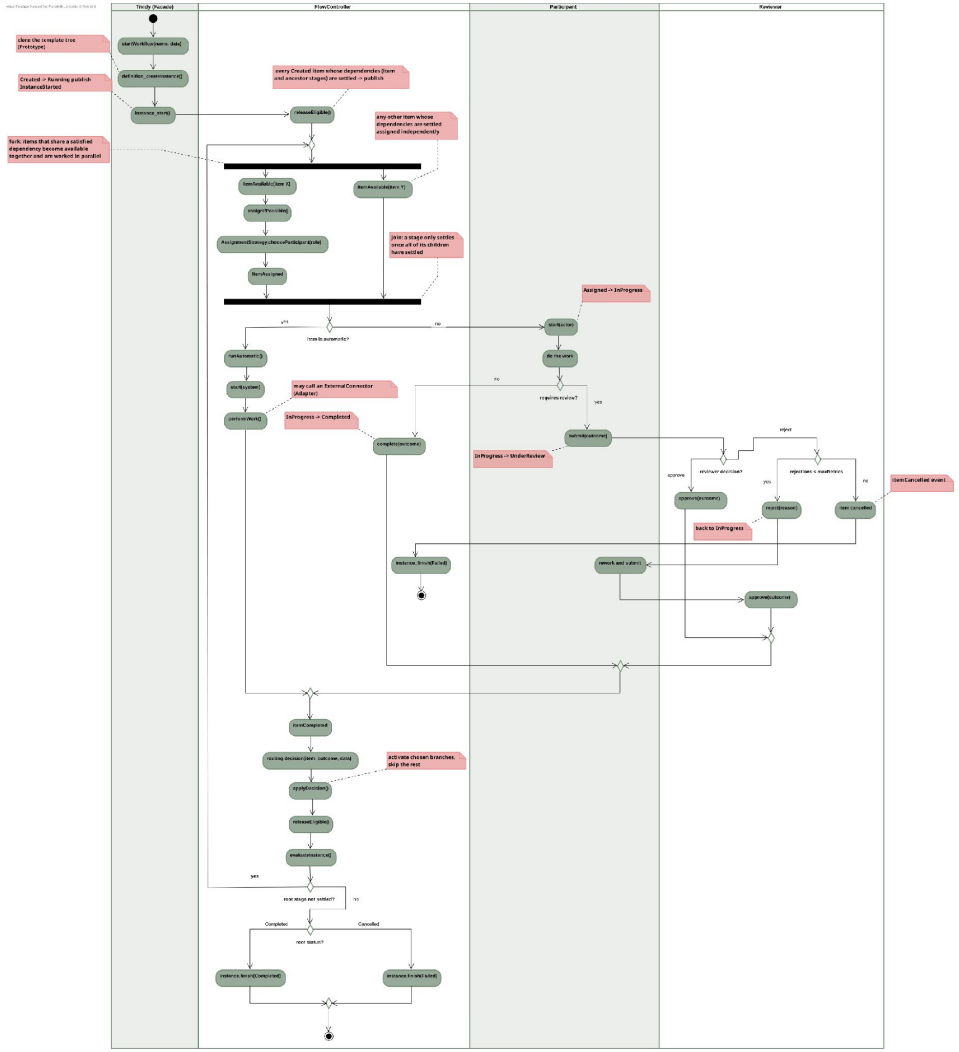
advanceTime moves the simulated clock · DeadlineMonitor sweeps the tree · Expire or escalate, then the chain of responsibility

Failure and escalation



A technical failure is retried when retryable · Otherwise the item escalates up the chain · If nobody resolves it, it stays Escalated

The engine loop



KEY POINTS

- Every change is an event
- FlowController routes, assigns and runs automatic items, then evaluates the instance

Key facts about our implementation

Why simulated time?

ManualClock and advanceTime(seconds) make deadlines demonstrable live and testable in milliseconds.

Expire or escalate?

A task's timeoutCode() decides: with a code the item expires with that outcome; otherwise it escalates up the chain.

Can everything be undone?

Undo restores a snapshot. Work that cannot be reversed reports isReversible() as false.

Do runs affect each other?

No. Each instance is a deep clone with its own data, events, audit log and history.

What if a definition changes?

A running instance keeps the version it started with (definitionVersion()).

What if an outside system fails?

The failure is caught, ItemFailed is published, and a retryable failure can be retried; otherwise it escalates.

Where things live in the repository

Where	What is there
`include/engine, src/engine`	The generic engine: facade, definitions, instances, items, policies, events, commands. No industry names.
`include/industries`	The three packs: banking (Hudson), telecom (Warped) and the document-approval demo.
`include/ui, app/main.cpp`	The TUI and client profiles. main.cpp installs the three packs and starts the TUI.
`tests`	GoogleTest suites for the engine, both industries and the UI.
`docs`	Diagrams, research material and report assets.
`Dockerfile, .devcontainer`	The pinned toolchain, and the Codespaces dev container.
`.github/workflows`	CI, branch-name, PR-title, label and Doxygen workflows.

04

How we built it

GitHub workflow, testing, CI, Docker and Doxygen: what we did and why.

From branch to main: review and automation



Main is always releasable

A red check blocks the merge, so main can never be broken by a PR.

Reviewed by the team

Team approvals per PR means several people have read every change.

Rules are automatic

Names, titles and labels are enforced by workflows, not by nagging.

How merging to main helped us

THE USUAL SCHOOL-PROJECT WAY

- A long-lived dev branch collects everyone's work
- CI and review are skipped along the way
- One giant PR into main at the end
- Bugs and conflicts all surface in the final days

OUR WAY (HYBRID APPROACH)

- Small PRs go straight to protected main
- CI builds and tests every one against main
- Team approvals, so review is continuous
- Main stays green; there is no big-bang merge
- Work done in close collaboration is done in an integration branch

CI made it safe: a branch that is always tested and reviewed does not need a second branch to hide on.

Tests first, and tests that prove the requirements

1

RED

Write a failing test for the behaviour

2

GREEN

Write the least code that passes

3

REFACTOR

Tidy up with the tests as a safety net

then repeat for the next behaviour, this is TDD

WHAT THE SUITE COVERS (GoogleTest)

- Engine core, items and the state machine
- Events, policies and the runtime (routing, deadlines)
- Connectors and workflow-definition validation
- Both industries: banking and telecom
- The UI layer
- Functional requirement testing

Transitions

every legal and illegal action

Routing

branches and skips

Isolation

instances never leak

CI: what runs on every pull request

Build and test

Builds the Docker image, then runs the full test suite inside it. Red blocks the merge.

Branch name check

Branches must follow the agreed naming pattern.

PR title check + labels

Titles follow the convention and PRs are labelled automatically.

Valgrind leak check

make valgrind fails on definite or indirect leaks.

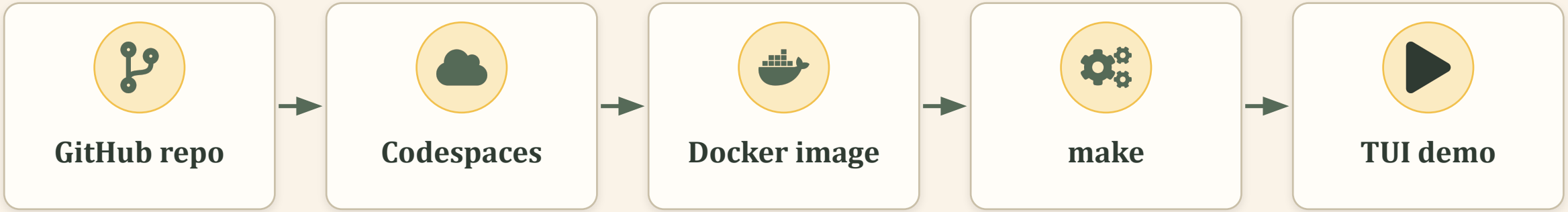
Doxygen on Pages

Docs redeploy to GitHub Pages on every merge to main.

Team approvals

Branch protection: nobody merges alone.

Docker: the same toolchain everywhere



WHAT THE IMAGE PINS

- Ubuntu 22.04 and GCC 11
- GoogleTest 1.12.1 (1.13 dropped C++11)
- Doxygen and valgrind for docs and leak checks
- A dev container with docker-in-docker for Codespaces

WHY DOCKER

- Accessibility: nothing to install. Anyone can run the project from any computer.
- One environment for CI, laptops and Codespaces, so “works on my machine” disappears.
- This demo runs from Codespaces, using exactly the same image.

Doxygen: documentation that stays current



Written in the code

Public headers carry Doxygen comments: each class's role, its collaborators and the preconditions of its methods.



Published automatically

A workflow regenerates the HTML and publishes it to GitHub Pages on every merge to main.



Always matches main

The site at jaymacaskill.github.io/tividy is rebuilt from the code, so it cannot drift from it.

Documentation is part of the pipeline: nobody has to remember to publish it.

Out of everything we covered: our proudest bonus work

CI on every pull request	GitHub Actions builds the Docker image and runs the whole test suite; a red PR cannot merge.
Docker and dev container	One toolchain everywhere; make docker-run needs nothing installed.
Valgrind leak checks	make valgrind fails on definite or indirect leaks.
Doxygen on GitHub Pages	API docs redeploy automatically on every merge to main.
Automated PR hygiene	Branch-name and PR-title checks, automatic labels, a PR template and team approvals.
Simulated clock	Deadlines are demonstrable live and testable in milliseconds.

Questions?

16

design patterns

2 + 1

industries and a demo pack

0

engine edits per industry

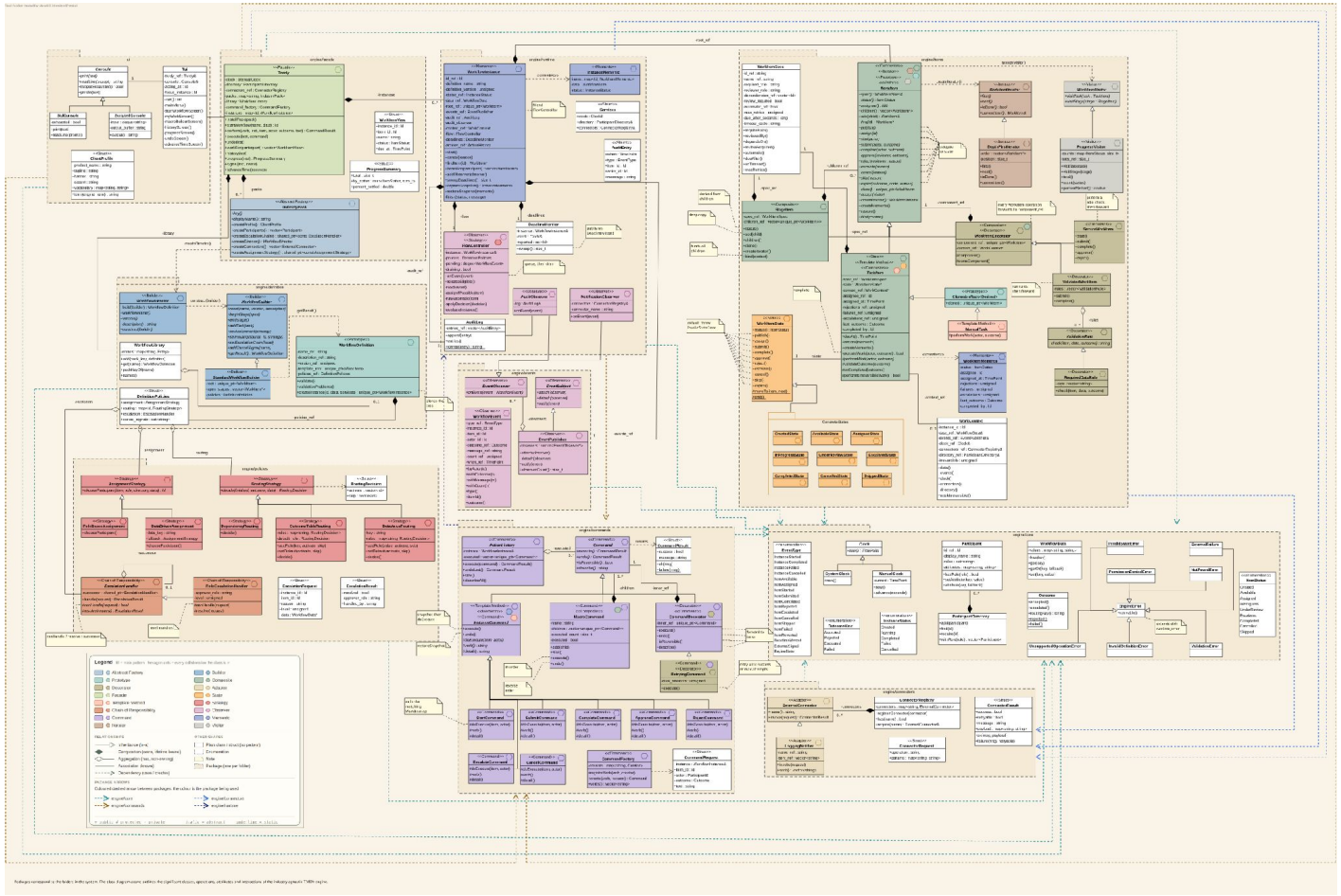
TIVIDY · Team Snack Overflow

A

Appendix: class diagrams

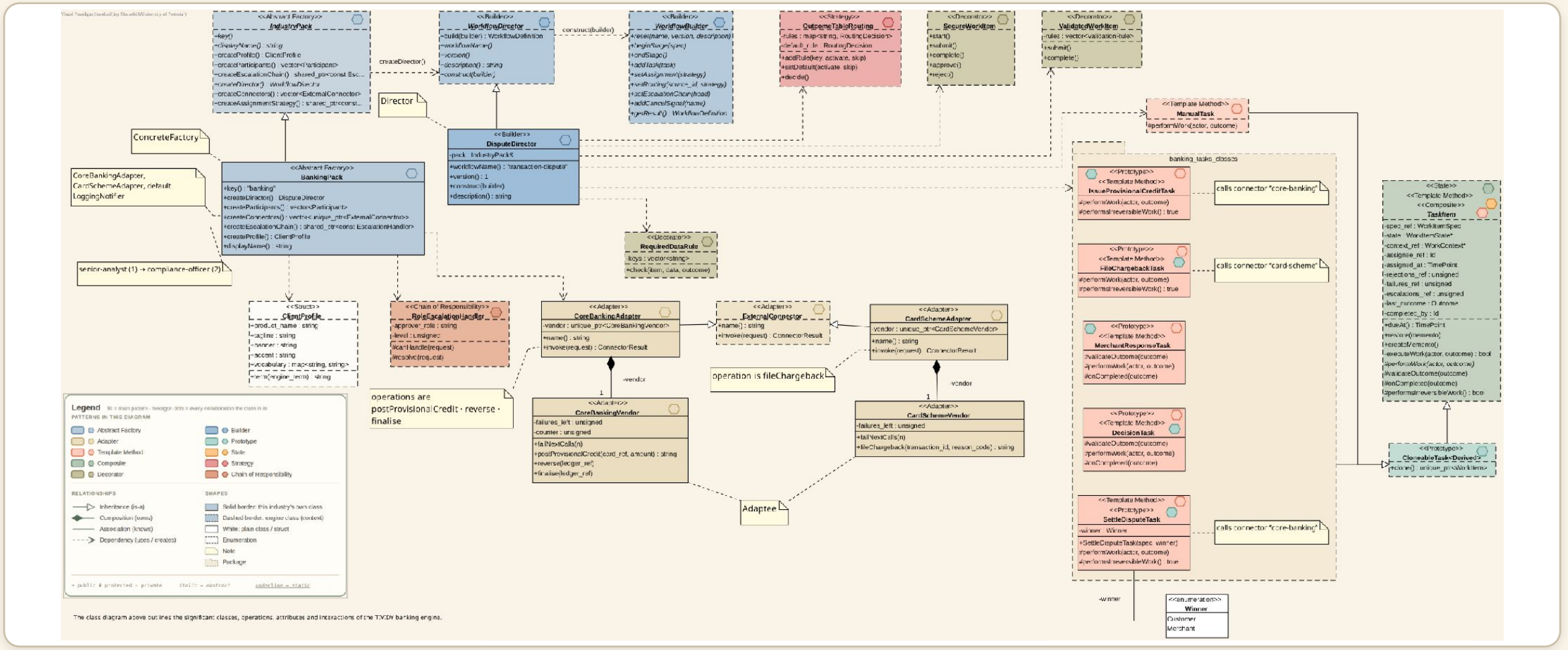
Full engine and industry class diagrams (uploaded to the repository as .svg and .vpp files).

Engine class diagram (full)



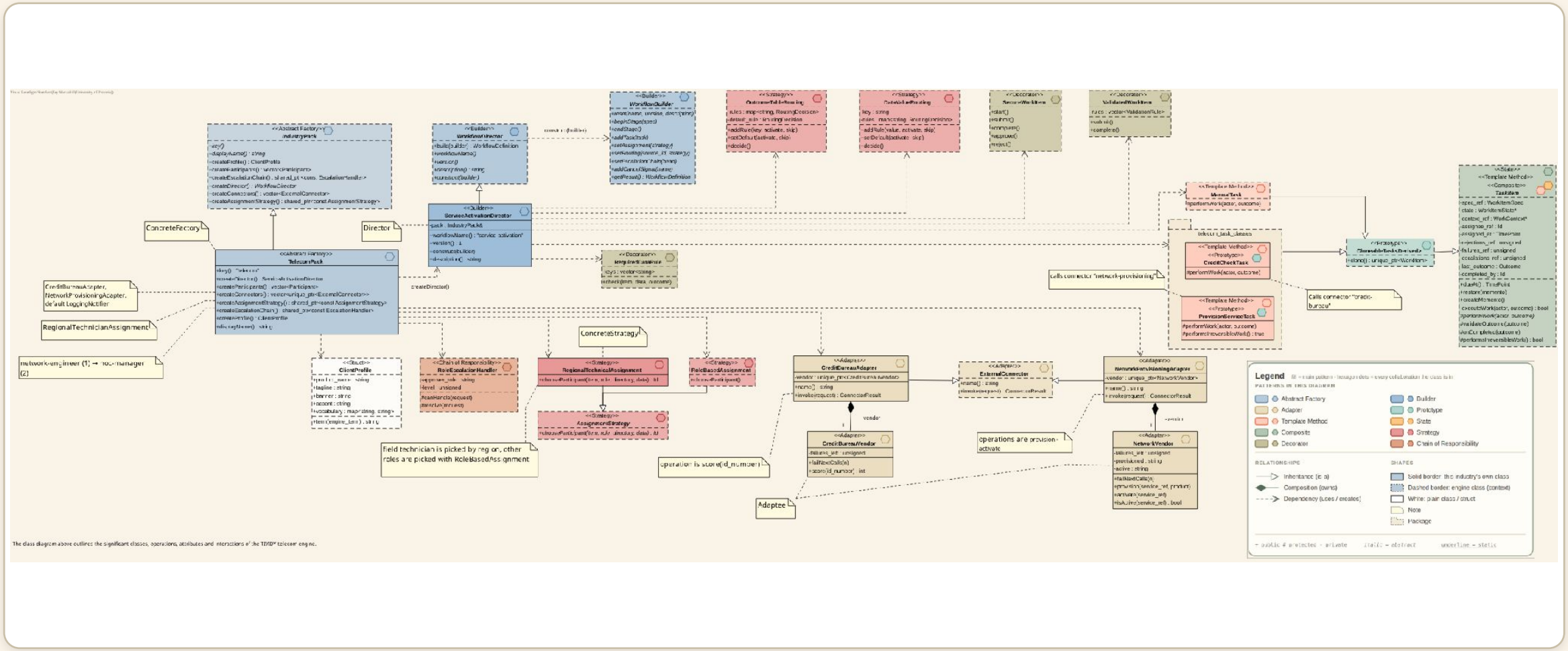
The industry-neutral engine, with every class · Colour marks which design pattern a class belongs to

Hudson class diagram



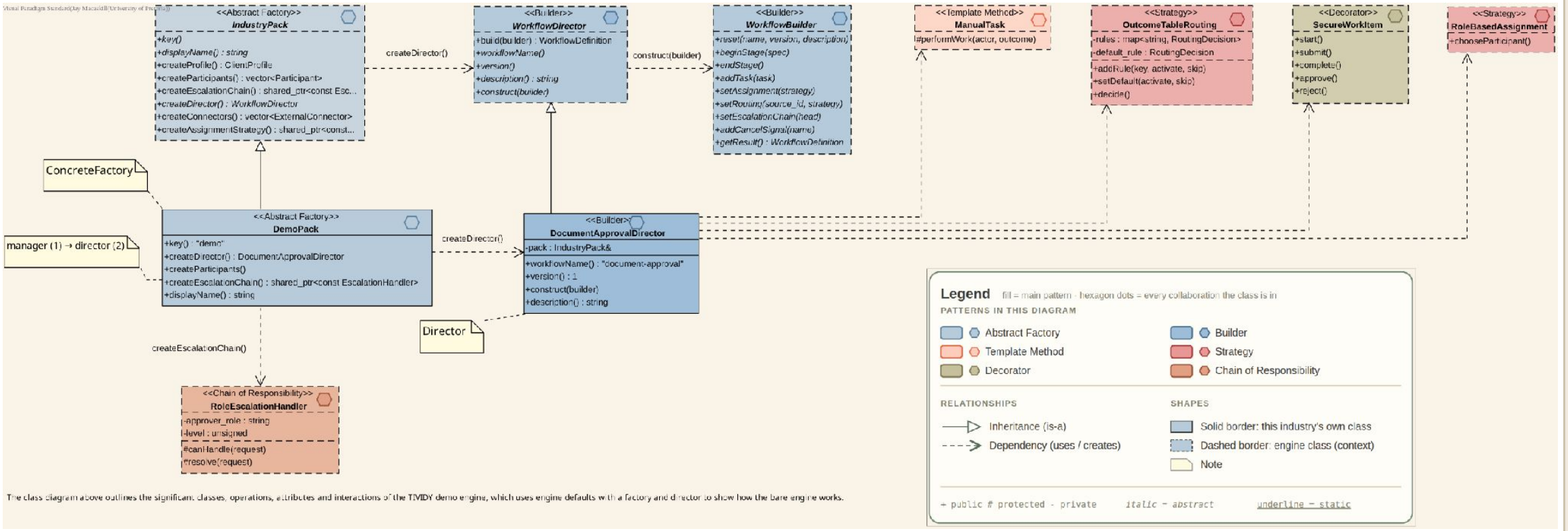
Solid border: the industry's own class · Dashed border: an engine class shown for context

Warped class diagram



Same engine classes, different pack · Regional technician assignment lives only here

Document-approval class diagram



Uses only generic engine classes · Proves the engine is generic